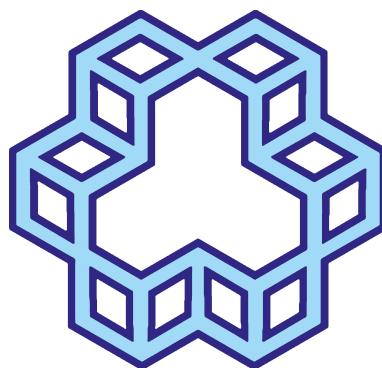




دانشگاه صنعتی خواجه نصیرالدین طوسی
دانشکده مهندسی برق - گروه مهندسی کنترل

Fundamentals of Artificial Intelligence

گزارش پروژه نهایی

FIS4041 Final Project

نام و نام خانوادگی	نیکی مهدیان - مبینا یوسفی مقدم
شماره دانشجویی	۴۰۱۲۳۱۵۳ - ۴۰۱۲۴۰۹۳
استاد	دکتر علیاری
تاریخ	بهمن ماه ۱۴۰۴



فهرست مطالب

۷	۱	مقدمه
۷	۱.۱	ساختار پروژه
۷	۲.۱	لینک‌های پروژه
۷	۲	سوال ۱: انتخاب ویژگی با استفاده از الگوریتم‌های بهینه‌سازی
۷	۱.۲	مقدمه
۷	۲.۲	پیش‌پردازش داده
۸	۳.۲	الگوریتم بهینه‌سازی ازدحام ذرات (PSO)
۸	۱.۳.۲	مکانیزم الگوریتم PSO
۸	۲.۳.۲	پارامترهای الگوریتم PSO و تحلیل آن‌ها
۹	۳.۳.۲	تحلیل همگرایی PSO
۹	۴.۳.۲	نتایج PSO
۹	۴.۲	الگوریتم ژنتیک (GA)
۹	۱.۴.۲	مکانیزم الگوریتم GA
۱۰	۲.۴.۲	پارامترهای الگوریتم GA و تحلیل آن‌ها
۱۰	۳.۴.۲	تحلیل همگرایی GA
۱۰	۴.۴.۲	نتایج GA
۱۱	۵.۲	مقایسه نتایج
۱۱	۱.۵.۲	تحلیل عملکرد و مقایسه جامع
۱۲	۳	سوال ۲: خوشه‌بندی و کاهش ابعاد
۱۲	۱.۳	مقدمه
۱۳	۲.۳	پیش‌پردازش داده
۱۳	۳.۳	تحلیل مؤلفه اصلی (PCA)
۱۳	۴.۳	خوشه‌بندی K-Means
۱۳	۱.۴.۳	تحلیل جامع K از ۲ تا ۱۰
۱۴	۲.۴.۳	انتخاب K بهینه
۱۵	۵.۳	خوشه‌بندی سلسله‌مراتبی تجمعی
۱۵	۱.۵.۳	مقایسه روش‌های پیوند
۱۶	۲.۵.۳	بهترین روش پیوند
۱۶	۶.۳	خوشه‌بندی DBSCAN
۱۶	۱.۶.۳	جستجوی شبکه پارامترها
۱۷	۲.۶.۳	بهترین پیکربندی DBSCAN
۱۸	۷.۳	تصاویر خوشه‌بندی
۱۸	۱.۷.۳	تحلیل معیارهای خوشه‌بندی



۱۸	نمودار Inertia و Score Silhouette برای K-Means	۲.۷.۳
۱۹	مقایسه Score Silhouette برای روش‌های پیوند Agglomerative	۳.۷.۳
۲۰	نقشه حرارتی Score Silhouette برای پارامترهای DBSCAN	۴.۷.۳
۲۱	مقایسه بهترین امتیاز Silhouette برای سه الگوریتم	۵.۷.۳
۲۲	سوال ۳: نظریه یادگیری Q	۴
۲۲	مقدمه	۱.۴
۲۲	بخش الف: معادله و پارامترهای یادگیری Q	۲.۴
۲۲	معادله یادگیری Q	۱.۲.۴
۲۳	پارامترهای الگوریتم	۲.۲.۴
۲۳	یادگیری Off-Policy	۳.۲.۴
۲۳	بخش ب: محاسبه مقدار Q	۳.۴
۲۴	بخش ج: مسائل همگرایی و راه‌حل‌ها	۴.۴
۲۴	نرخ یادگیری نامناسب (α)	۱.۴.۴
۲۴	اکتشاف ناکافی یا استراتژی اکتشاف ضعیف	۲.۴.۴
۲۴	پروژه نهایی: خط لوله جامع یادگیری ماشین	۵
۲۴	مقدمه	۱.۵
۲۴	بخش ۱: معرفی مسئله و داده	۲.۵
۲۴	بخش ۲: پیش‌پردازش داده	۳.۵
۲۵	مدیریت مقادیر گم‌شده	۱.۳.۵
۲۵	کدگذاری متغیرهای دسته‌ای	۲.۳.۵
۲۵	تشخیص و درمان داده‌های پرت	۳.۳.۵
۲۶	مقیاس‌گذاری ویژگی‌ها	۴.۳.۵
۲۶	بخش ۳: کاهش ابعاد	۴.۵
۲۶	PCA (تحلیل مؤلفه اصلی)	۱.۴.۵
۲۶	LDA (تحلیل تشخیص خطی)	۲.۴.۵
۲۶	t-SNE	۳.۴.۵
۲۷	بخش ۴: انتخاب و آموزش مدل	۵.۵
۲۷	Random Forest	۱.۵.۵
۲۷	Gradient Boosting	۲.۵.۵
۲۸	Support Vector Machine	۳.۵.۵
۲۸	بخش ۵: تنظیم ابرپارامترها	۶.۵
۲۸	Random Forest	۱.۶.۵
۲۹	Gradient Boosting	۲.۶.۵
۲۹	بخش ۶: ارزیابی و قابلیت تکرار	۷.۵
۲۹	نتایج نهایی و تحلیل جامع	۱.۷.۵



۲۹	تحلیل مشکلات و مسائل	۲.۷.۵
۳۰	پیشنهادهای بهبود	۳.۷.۵
۳۰	بخش ۷: تجسم و تحلیل	۸.۵
۳۰	ماتریس سردرگمی	۱.۸.۵
۳۱	منحنی ROC	۲.۸.۵
۳۳	منحنی Precision-Recall	۳.۸.۵
۳۴	مقایسه معیارها	۴.۸.۵
۳۵	اهمیت ویژگی‌ها	۵.۸.۵
۳۶	توزیع احتمالات	۶.۸.۵
۳۶	خلاصه مقایسه مدل‌ها	۷.۸.۵
۳۷	نتیجه‌گیری	۶



فهرست تصاویر

۱	مقایسه عملکرد الگوریتم‌های PSO و GA شامل نمودار همگرایی، مقایسه دقت و تعداد ویژگی‌های انتخاب شده	۱۱
۲	نمایش بصری خوشه‌بندی‌های مختلف در فضای دو بعدی PCA	۱۸
۳	نمودار Inertia و Score Silhouette برای K-Means با تعداد خوشه‌های مختلف از K=2 تا K=10	۱۹
۴	مقایسه Score Silhouette برای چهار روش پیوند مختلف در الگوریتم Clustering Agglomerative	۲۰
۵	نقشه حرارتی Score Silhouette برای ترکیبات مختلف پارامترهای eps و min_samples در الگوریتم DBSCAN	۲۱
۶	مقایسه بهترین امتیاز Silhouette برای هر یک از سه الگوریتم خوشه‌بندی	۲۲
۷	ماتریس سردرگمی برای مدل Forest Random	۳۱
۸	ماتریس سردرگمی برای مدل Boosting Gradient	۳۲
۹	منحنی ROC برای مقایسه عملکرد مدل‌ها	۳۲
۱۰	منحنی Precision-Recall برای مقایسه عملکرد مدل‌ها	۳۳
۱۱	نمودار مقایسه معیارهای مختلف برای دو مدل	۳۴
۱۲	نمودار اهمیت ویژگی‌ها از مدل Forest Random	۳۵
۱۳	توزیع احتمالات پیش‌بینی برای مدل Forest Random	۳۶
۱۴	توزیع احتمالات پیش‌بینی برای مدل Boosting Gradient	۳۷



فهرست جداول

۱۲	 مقایسه الگوریتم‌های PSO و GA	۱
۱۴	 نتایج K-Means برای K های مختلف	۲
۱۵	 مقایسه روش‌های پیوند در خوشه‌بندی سلسله‌مراتبی	۳
۱۷	 مقایسه عملکرد الگوریتم‌های خوشه‌بندی	۴
۲۵	 نتایج تشخیص داده‌های پرت	۵



فهرست برنامه‌ها



۱ مقدمه

این گزارش شامل پیاده‌سازی و تحلیل چهار بخش اصلی پروژه نهایی درس مبانی هوش مصنوعی (FIS4041) است. این پروژه شامل انتخاب ویژگی با استفاده از الگوریتم‌های بهینه‌سازی، خوشه‌بندی داده‌ها، بررسی نظری یادگیری تقویتی و پیاده‌سازی کامل یک خط لوله یادگیری ماشین است.

۱.۱ ساختار پروژه

پروژه شامل چهار بخش اصلی است که شامل سوال ۱ با انتخاب ویژگی با استفاده از الگوریتم‌های PSO و GA، سوال ۲ با خوشه‌بندی داده‌ها با استفاده از Agglomerative، K-Means و DBSCAN، سوال ۳ با بررسی نظری الگوریتم یادگیری Q (Q-Learning) و پروژه نهایی با پیاده‌سازی کامل خط لوله یادگیری ماشین شامل پیش‌پردازش، کاهش ابعاد، انتخاب مدل، تنظیم ابرپارامترها و ارزیابی می‌شود.

۲.۱ لینک‌های پروژه

کدها و فایل‌های پروژه در مخزن گیت‌هاب و درایو گوگل در دسترس است:

- مخزن گیت‌هاب: <https://github.com/nikimahdian/Principles-of-Intelligent-Systems-4041>

- درایو گوگل: <https://drive.google.com/drive/folders/1xLn2NnYwpyvsRD6pQqpk7KczZQcJD2Jg>

۲ سوال ۱: انتخاب ویژگی با استفاده از الگوریتم‌های بهینه‌سازی

۱.۲ مقدمه

در این بخش، انتخاب ویژگی با استفاده از دو الگوریتم بهینه‌سازی انجام شده است: الگوریتم بهینه‌سازی ازدحام ذرات (Particle Swarm Optimization یا PSO) و الگوریتم ژنتیک (Genetic Algorithm یا GA). هدف از انتخاب ویژگی، یافتن زیرمجموعه‌ای از ویژگی‌ها است که بهترین عملکرد را در پیش‌بینی وضعیت وام داشته باشد. انتخاب ویژگی یک مسئله بهینه‌سازی ترکیبی است که در آن باید زیرمجموعه‌ای از ویژگی‌ها را انتخاب کنیم که هم دقت مدل را به حداکثر برساند و هم تعداد ویژگی‌ها را به حداقل برساند.

۲.۲ پیش‌پردازش داده

داده‌های مجموعه Loan Dataset بارگذاری و پیش‌پردازش شدند. مجموعه داده شامل 349 نمونه آموزشی و 150 نمونه تست است که با استفاده از تقسیم 70-30 به روش stratified split ایجاد شده‌اند. در مجموع 11 ویژگی در این مجموعه داده وجود دارد که شامل ویژگی‌های دسته‌ای و عددی می‌شوند.

ویژگی‌های دسته‌ای شامل جنسیت متقاضی (Gender)، وضعیت تأهل (Married)، تعداد وابستگان (Dependents)، سطح تحصیلات (Education)، وضعیت خوداشتغالی (Self_Employed) و منطقه ملک (Area) هستند. ویژگی‌های عددی شامل درآمد متقاضی (Applicant_Income)،



درآمد هم‌متقاضی (Coapplicant_Income)، مبلغ وام درخواستی (Loan_Amount)، مدت زمان وام (Term) و تاریخچه اعتباری (Credit_History) می‌شوند.

برای آماده‌سازی داده‌ها، متغیرهای دسته‌ای با استفاده از LabelEncoder به فرمت عددی تبدیل شدند. تقسیم داده به صورت stratified انجام شد تا توزیع کلاس‌ها در مجموعه‌های آموزشی و تست حفظ شود. این روش تضمین می‌کند که نسبت کلاس‌های مثبت و منفی در هر دو مجموعه یکسان باشد و از این طریق نتایج قابل اعتمادتری حاصل می‌شود.

۳.۲ الگوریتم بهینه‌سازی ازدحام ذرات (PSO)

الگوریتم PSO از هوش جمعی استفاده می‌کند که در آن ذرات در فضای جستجو حرکت می‌کنند تا ترکیبات بهینه ویژگی‌ها را پیدا کنند. این الگوریتم از رفتار جمعی پرندگان یا ماهی‌ها الهام گرفته شده است.

۱.۳.۲ مکانیزم الگوریتم PSO

در الگوریتم PSO، هر ذره نشان‌دهنده یک راه‌حل احتمالی است که در این مسئله به معنای یک ترکیب از ویژگی‌های انتخاب شده است. هر ذره دارای سه ویژگی اصلی است: موقعیت فعلی که نشان‌دهنده ترکیب فعلی ویژگی‌های انتخاب شده است، سرعت که جهت و میزان حرکت در فضای جستجو را تعیین می‌کند، و بهترین موقعیت شخصی که بهترین ترکیب ویژگی‌هایی است که این ذره تاکنون پیدا کرده است.

حرکت ذرات در فضای جستجو تحت تأثیر سه مؤلفه اصلی قرار می‌گیرد. مؤلفه اول اینرسی است که تمایل ذره به ادامه حرکت در جهت فعلی را نشان می‌دهد. مؤلفه دوم شناختی است که ذره را به سمت بهترین موقعیت شخصی خود هدایت می‌کند. مؤلفه سوم اجتماعی است که تمایل ذره به حرکت به سمت بهترین موقعیت کل جمعیت را کنترل می‌کند. ترکیب این سه مؤلفه باعث می‌شود که ذرات به طور همزمان در فضای جستجو کاوش کنند و از تجربیات شخصی و جمعی خود استفاده کنند.

۲.۳.۲ پارامترهای الگوریتم PSO و تحلیل آن‌ها

الگوریتم PSO با تعداد 50 ذره اجرا شد که تعادل مناسبی بین کیفیت جستجو و سرعت محاسبات ایجاد می‌کند. تعداد ذرات بیشتر منجر به جستجوی جامع‌تر می‌شود اما هزینه محاسباتی را افزایش می‌دهد. تعداد تکرارها 100 در نظر گرفته شد که برای همگرایی الگوریتم کافی است. با توجه به نمودار همگرایی، الگوریتم در حدود تکرار 7 به جواب بهینه رسیده است که نشان‌دهنده سرعت بالای همگرایی است.

وزن اینرسی (w) برابر 0.9 تنظیم شد که میزان تأثیر سرعت قبلی بر حرکت فعلی را کنترل می‌کند. مقدار 0.9 نشان‌دهنده اینرسی بالاست که به ذرات اجازه می‌دهد در فضای جستجو به خوبی کاوش کنند. این مقدار برای مراحل اولیه جستجو مناسب است و به ذرات کمک می‌کند تا مناطق مختلف فضای جستجو را بررسی کنند.

پارامتر شناختی (c1) برابر 0.5 تنظیم شد که تأثیر بهترین موقعیت شخصی هر ذره را کنترل می‌کند. این مقدار تعادل مناسبی بین اکتشاف شخصی و اجتماعی ایجاد می‌کند. پارامتر اجتماعی (c2) برابر 0.3 تنظیم شد که کمتر از c1 است و نشان می‌دهد الگوریتم بیشتر بر اکتشاف شخصی تکیه دارد. این ترکیب که در آن c1 بزرگ‌تر از c2 است برای مسائل انتخاب ویژگی مناسب است چون به هر ذره اجازه می‌دهد تا به طور مستقل فضای جستجو را کاوش کند.



پارامتر α برابر 0.7 تنظیم شد که وزن تعادل بین دقت و تعداد ویژگی‌ها در تابع هدف را تعیین می‌کند. فرمول تابع هدف به صورت $J = \alpha(1 - Acc) + (1 - \alpha)(selected_features/total_features)$ است. با $\alpha = 0.7$ ، دقت 70% و تعداد ویژگی‌ها 30% وزن دارند که نشان می‌دهد اولویت بیشتری به دقت داده شده است تا به حداقل رساندن تعداد ویژگی‌ها.

۳.۳.۲ تحلیل همگرایی PSO

با بررسی تاریخچه هزینه (cost_history) می‌توان روند همگرایی الگوریتم را مشاهده کرد. در تکرار اول، هزینه اولیه 0.2444 بود که نشان‌دهنده کیفیت پایین راه‌حل‌های اولیه است. الگوریتم به سرعت بهبود یافت و در تکرار 7 به هزینه بهینه 0.1626 رسید. در تکرارهای بعدی، هزینه در مقدار 0.1626 تثبیت شد اما نوسانات جزئی وجود داشت. در برخی تکرارها هزینه به 0.1899 یا 0.1992 افزایش یافت که نشان‌دهنده اکتشاف مداوم فضای جستجو است و به الگوریتم کمک می‌کند تا از بهینه‌های محلی فرار کند. نکته مهم این است که الگوریتم در تکرار 7 به جواب بهینه رسید و در 93 تکرار باقی‌مانده این جواب را حفظ کرد. این رفتار نشان می‌دهد که الگوریتم به سرعت همگرا شد و جواب پیدا شده پایدار است. با توجه به این که الگوریتم در تکرارهای اولیه به جواب بهینه می‌رسد، می‌توان تعداد تکرارها را کاهش داد تا سرعت محاسبات افزایش یابد بدون این که کیفیت جواب تحت تأثیر قرار گیرد.

۴.۳.۲ نتایج PSO

الگوریتم PSO موفق شد از بین 11 ویژگی موجود، تنها یک ویژگی را انتخاب کند که کاهش 90.9 درصدی در تعداد ویژگی‌ها را نشان می‌دهد. ویژگی انتخاب شده Credit_History با شاخص 9 است که تاریخچه اعتباری متقاضی را نشان می‌دهد. با استفاده از این ویژگی واحد، الگوریتم به دقت نهایی 0.8067 معادل 80.67% دست یافت. بهترین هزینه به دست آمده 0.1626 است و الگوریتم در مدت 7 تکرار به این جواب بهینه رسید.

تحلیل انتخاب ویژگی Credit_History نشان می‌دهد که این ویژگی مهم‌ترین عامل پیش‌بینی کننده وضعیت وام است. با تنها این ویژگی می‌توان به دقت 80.67% دست یافت که نشان‌دهنده قدرت پیش‌بینی بالای این ویژگی است. این نتیجه نشان می‌دهد که سایر ویژگی‌ها اطلاعات اضافی کمی دارند یا با Credit_History همبستگی بالایی دارند و بنابراین در پیش‌بینی نقش مهمی ایفا نمی‌کنند. مدل تک‌ویژگی حاصل شده بسیار قابل تفسیر است و سادگی آن باعث می‌شود که برای استقرار در محیط عملیاتی مناسب باشد.

۴.۲ الگوریتم ژنتیک (GA)

الگوریتم GA از اصول تکاملی استفاده می‌کند: انتخاب، تقاطع و جهش برای تکامل زیرمجموعه‌های ویژگی. این الگوریتم از فرآیند تکامل طبیعی الهام گرفته شده است.

۱.۴.۲ مکانیزم الگوریتم GA

در الگوریتم GA، هر فرد که به آن کروموزوم گفته می‌شود نشان‌دهنده یک ترکیب از ویژگی‌ها است. نمایش هر کروموزوم به صورت یک رشته باینری است که در آن 1 به معنای انتخاب ویژگی و 0 به معنای عدم انتخاب است. تابع برازش بر اساس دقت مدل با استفاده از ویژگی‌های انتخاب شده محاسبه می‌شود. عملگرهای ژنتیک شامل سه عملگر اصلی هستند: انتخاب که والدین برتر را برای تولید نسل بعد انتخاب می‌کند، تقاطع که اطلاعات والدین را ترکیب می‌کند تا فرزندان جدید ایجاد شود، و جهش که تغییر تصادفی برای حفظ تنوع در جمعیت ایجاد می‌کند.



۲.۴.۲ پارامترهای الگوریتم GA و تحلیل آن‌ها

اندازه جمعیت برابر 50 تنظیم شد که تعداد افراد در هر نسل را مشخص می‌کند. جمعیت بزرگ‌تر منجر به تنوع بیشتر می‌شود اما هزینه محاسباتی را افزایش می‌دهد و 50 فرد تعادل مناسبی بین کیفیت و سرعت ایجاد می‌کند. تعداد نسل برابر 50 در نظر گرفته شد که تعداد نسل‌های تکامل را مشخص می‌کند. با بررسی تاریخچه برازش، الگوریتم در نسل 3 به جواب بهینه رسید که نشان می‌دهد 50 نسل برای اطمینان از همگرایی کافی است.

احتمال تقاطع برابر 0.9 تنظیم شد که احتمال انجام تقاطع بین دو والد را مشخص می‌کند. این مقدار یعنی 90% از فرزندان از تقاطع تولید می‌شوند که به الگوریتم اجازه می‌دهد به سرعت ترکیبات خوب را کشف کند. استفاده از تقاطع دو نقطه (two-point crossover) برای تبادل بهتر اطلاعات بین والدین انجام می‌شود. احتمال جهش برابر 0.1 تنظیم شد که احتمال تغییر هر بیت در کروموزوم را مشخص می‌کند. این مقدار یعنی 10% احتمال جهش برای هر بیت که برای حفظ جواب‌های خوب و جلوگیری از تخریب تصادفی مناسب است. جهش با استفاده از bit flip mutation انجام می‌شود که بیت‌های کروموزوم را به صورت تصادفی تغییر می‌دهد.

روش انتخاب Tournament Selection است که افراد برتر را بر اساس مسابقه انتخاب می‌کند. این روش فشار انتخابی مناسبی ایجاد می‌کند و به افراد با برازش بهتر شانس بیشتری برای تولید مثل می‌دهد. پارامتر α برابر 0.7 تنظیم شد که همان پارامتر تعادل در تابع هدف است که در PSO نیز استفاده شد و برای مقایسه عادلانه بین دو الگوریتم ضروری است.

۳.۴.۲ تحلیل همگرایی GA

با بررسی تاریخچه برازش (fitness_history) می‌توان روند همگرایی الگوریتم را مشاهده کرد. در نسل اول، برازش متوسط 0.3254 بود که نشان‌دهنده کیفیت پایین راه‌حل‌های اولیه است. در نسل دوم، برازش متوسط به 0.2817 کاهش یافت که نشان‌دهنده بهبود است. در نسل سوم، الگوریتم به برازش بهینه 0.1626 رسید که نشان می‌دهد الگوریتم به سرعت همگرا شد. در نسل‌های بعدی، برازش در محدوده 0.1626 تا 0.1973 نوسان کرد اما برازش بهترین فرد از نسل 3 به بعد در 0.1626 تثبیت شد.

مقایسه برازش متوسط و بهترین نشان می‌دهد که برازش متوسط از 0.3254 به حدود 0.163-0.168 کاهش یافت که نشان می‌دهد کل جمعیت به سمت جواب بهینه حرکت کرده است. نوسانات جزئی در برازش متوسط نشان‌دهنده اکتشاف مداوم فضای جستجو است که به الگوریتم کمک می‌کند تا از بهینه‌های محلی فرار کند.

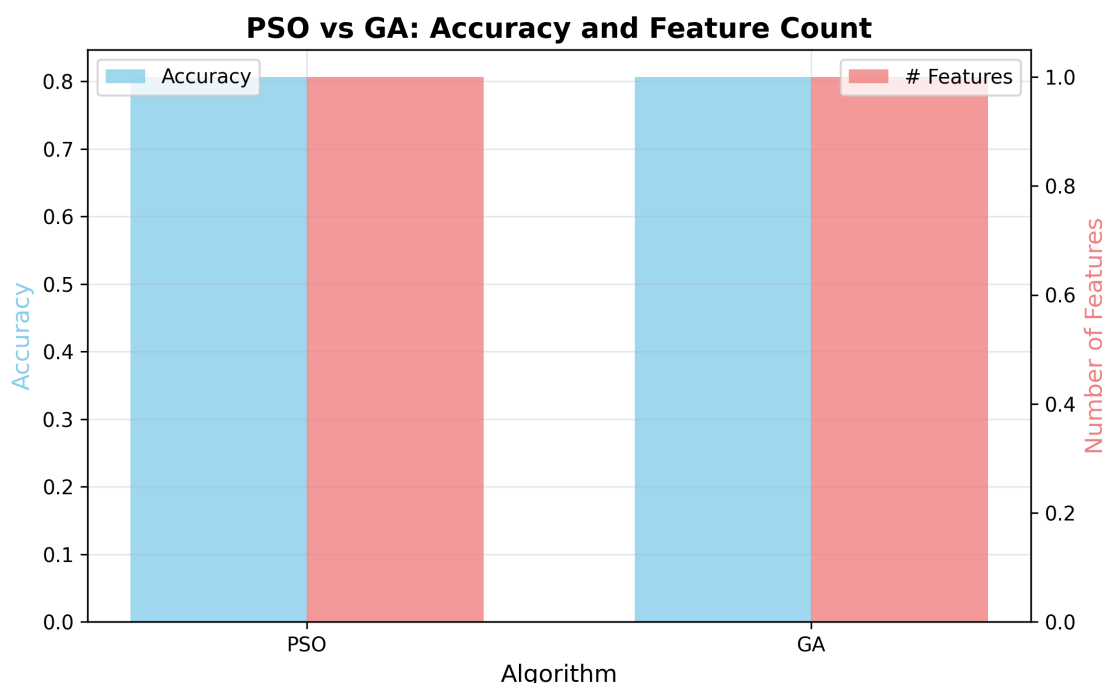
نکته مهم این است که الگوریتم GA در نسل 3 به جواب بهینه رسید که سریع‌تر از PSO است (نسل 3 در مقابل تکرار 7). این نشان می‌دهد که GA در این مسئله سریع‌تر همگرا می‌شود و عملگرهای ژنتیک شامل تقاطع و جهش به خوبی کار می‌کنند. با توجه به این که الگوریتم در نسل‌های اولیه به جواب بهینه می‌رسد، می‌توان تعداد نسل‌ها را کاهش داد تا سرعت محاسبات افزایش یابد.

۴.۴.۲ نتایج GA

الگوریتم GA موفق شد از بین 11 ویژگی موجود، تنها یک ویژگی را انتخاب کند که کاهش 90.9 درصدی در تعداد ویژگی‌ها را نشان می‌دهد. ویژگی انتخاب شده Credit_History با شاخص 9 است که تاریخچه اعتباری متقاضی را نشان می‌دهد. با استفاده از این ویژگی واحد، الگوریتم به دقت نهایی 0.8067 معادل 80.67% دست یافت. بهترین برازش به دست آمده 0.1626 است و الگوریتم در مدت 3 نسل به این جواب بهینه رسید که نشان‌دهنده سرعت بالای همگرایی است.

تحلیل انتخاب ویژگی نشان می‌دهد که الگوریتم GA نیز همان ویژگی Credit_History را انتخاب کرد که تأیید می‌کند این ویژگی واقعاً مهم‌ترین عامل پیش‌بینی است. همگرایی سریع‌تر GA در مقایسه با PSO نشان می‌دهد که عملگرهای ژنتیک شامل تقاطع و جهش برای این مسئله خاص مؤثرتر هستند و به الگوریتم کمک می‌کنند تا به سرعت به جواب بهینه برسد.

۵.۲ مقایسه نتایج



شکل ۱: مقایسه عملکرد الگوریتم‌های PSO و GA شامل نمودار همگرایی، مقایسه دقت و تعداد ویژگی‌های انتخاب شده

۱.۵.۲ تحلیل عملکرد و مقایسه جامع

هر دو الگوریتم به دقت یکسان ۸۰۶۷.۰ دست یافتند و هر دو ویژگی Credit_History را به عنوان مهم‌ترین ویژگی انتخاب کردند. این نتیجه مشترک نشان می‌دهد که تاریخچه اعتباری مهم‌ترین عامل در پیش‌بینی وضعیت وام است و کاهش ابعاد از ۱۱ ویژگی به ۱ ویژگی که معادل کاهش ۹۰.۹ درصدی است با حفظ دقت بالا امکان‌پذیر است. مدل تک‌ویژگی حاصل شده بسیار قابل تفسیر است و سادگی آن باعث می‌شود که برای استقرار در محیط عملیاتی مناسب باشد. همچنین این که هر دو الگوریتم به جواب یکسانی رسیدند نشان می‌دهد که جواب بهینه پایدار است و به الگوریتم خاصی بستگی ندارد.

مقایسه سرعت همگرایی نشان می‌دهد که الگوریتم PSO در تکرار ۷ به جواب بهینه رسید در حالی که الگوریتم GA در نسل ۳ به جواب بهینه رسید. بر این اساس به نظر می‌رسد GA سریع‌تر همگرا شده است اما این مقایسه کاملاً عادلانه نیست چون هر تکرار PSO شامل ارزیابی ۵۰ ذره است و هر نسل GA شامل ارزیابی ۵۰ فرد است اما محاسبات درونی این دو الگوریتم متفاوت است و نمی‌توان به طور مستقیم تعداد تکرارها را با تعداد نسل‌ها مقایسه کرد.

مقایسه پایداری نتایج نشان می‌دهد که الگوریتم PSO پس از رسیدن به جواب بهینه، نوسانات جزئی داشت و هزینه در محدوده ۱۶۲۶.۰ تا ۱۹۹۲.۰ تغییر کرد. در مقابل، الگوریتم GA پس از رسیدن به جواب بهینه، برازش بهترین فرد ثابت ماند اما برازش متوسط نوسان داشت که نشان‌دهنده اکتشاف مداوم فضای جستجو است. با این حال، هر دو الگوریتم جواب پایدار پیدا کردند و جواب بهینه در هر دو مورد حفظ شد.

مقایسه استراتژی جستجو نشان می‌دهد که الگوریتم PSO از جستجوی پیوسته در فضای ویژگی‌ها استفاده می‌کند و حرکت ذرات بر



اساس سرعت و موقعیت انجام می‌شود. این استراتژی برای مسائل پیوسته و گسسته مناسب است. در مقابل، الگوریتم GA از جستجوی گسسته با عملگرهای ژنتیک استفاده می‌کند که شامل تقاطع و جهش برای ایجاد تنوع است. این استراتژی برای مسائل گسسته و ترکیبی مناسب است و در این مسئله خاص که انتخاب ویژگی یک مسئله گسسته است، عملکرد بهتری داشته است.

مزایا و معایب:

معیار	PSO	GA
سرعت همگرایی	متوسط (۷ تکرار)	سریع (۳ نسل)
پایداری	خوب	خوب
پیچیدگی پیاده‌سازی	متوسط	متوسط-بالا
پارامترهای تنظیم	۵ پارامتر اصلی	۴ پارامتر اصلی
قابلیت تفسیر	متوسط	متوسط

جدول ۱: مقایسه الگوریتم‌های PSO و GA

تحلیل نمودار همگرایی نشان می‌دهد که هر دو الگوریتم به سرعت به جواب بهینه نزدیک شدند. الگوریتم GA زودتر به جواب رسید اما PSO نیز به سرعت همگرا شد. پس از رسیدن به جواب بهینه، هر دو الگوریتم آن را حفظ کردند اما نوسانات در PSO بیشتر است که نشان‌دهنده اکتشاف مداوم فضای جستجو است.

نتیجه‌گیری کلی این است که هر دو الگوریتم عملکرد یکسانی داشتند و به جواب یکسانی رسیدند. الگوریتم GA کمی سریع‌تر همگرا شد اما انتخاب بین آن‌ها به عوامل دیگری مانند پیاده‌سازی، پارامترهای موجود و ترجیحات بستگی دارد. مهم‌ترین نتیجه این تحلیل این است که ویژگی Credit_History به تنهایی می‌تواند ۸۰٪ دقت داشته باشد که نشان‌دهنده اهمیت بالای این ویژگی است.

با این حال، مشکلات احتمالی وجود دارد که باید مورد توجه قرار گیرد. انتخاب تنها یک ویژگی ممکن است نشان‌دهنده این باشد که سایر ویژگی‌ها اطلاعات کمی دارند یا پارامتر α خیلی بالا است و اولویت بیشتری به دقت داده شده است. برای بررسی این موضوع می‌توان α را کاهش داد تا الگوریتم ویژگی‌های بیشتری را انتخاب کند و ببینیم آیا دقت بهبود می‌یابد. همچنین دقت ۸۰٪ ممکن است برای برخی کاربردها کافی نباشد و در این صورت می‌توان از روش‌های دیگر مانند Recursive Feature Elimination یا Forward Selection استفاده کرد.

۳ سوال ۲: خوشه‌بندی و کاهش ابعاد

۱.۳ مقدمه

در این بخش، خوشه‌بندی داده‌های مشتریان مرکز خرید با استفاده از سه الگوریتم مختلف انجام شده است: الگوریتم K-Means Clustering که یک روش خوشه‌بندی مبتنی بر مرکز است، الگوریتم Agglomerative Hierarchical Clustering که یک روش خوشه‌بندی سلسله‌مراتبی است، و الگوریتم DBSCAN که یک روش خوشه‌بندی مبتنی بر چگالی است. هدف از این تحلیل، شناسایی الگوهای مشتریان و تقسیم آن‌ها به گروه‌های همگن برای استراتژی‌های بازاریابی هدفمند است.



۲.۳ پیش پردازش داده

داده‌های مشتریان مرکز خرید شامل ۲۰۰ نمونه و ۳ ویژگی عددی است که شامل سن مشتری (Age) بر حسب سال، درآمد سالانه (Annual Income) بر حسب هزار دلار، و امتیاز خرج کردن (Spending Score) در محدوده ۱ تا ۱۰۰ می‌شود. برای آماده‌سازی داده‌ها، همه ویژگی‌ها با استفاده از StandardScaler نرمال‌سازی شدند که بر اساس فرمول $z = \frac{x - \mu}{\sigma}$ عمل می‌کند. هدف از این نرمال‌سازی اطمینان از این است که همه ویژگی‌ها در مقیاس یکسان هستند که برای الگوریتم‌های مبتنی بر فاصله مانند K-Means و DBSCAN ضروری است. نکته مهم این است که تمام الگوریتم‌های خوشه‌بندی روی فضای اصلی ۳ بعدی آموزش داده شدند و PCA تنها برای تجسم ۲ بعدی استفاده شد.

۳.۳ تحلیل مؤلفه اصلی (PCA)

با استفاده از PCA دو بعدی برای تجسم، نسبت واریانس توضیح داده شده برای بعد اول ۴۴.۲۷٪ معادل ۲۷.۴۴٪ و برای بعد دوم ۳۳.۱۰٪ معادل ۳۱.۳۳٪ است. واریانس تجمعی در ۲ بعد برابر ۷۷.۵۷٪ معادل ۵۷.۷۷٪ است که نشان می‌دهد در مجموع ۵۷.۷۷٪ از واریانس در ۲ بعد حفظ شده است. این مقدار برای تجسم مناسب است اما ۴۳.۲۲٪ اطلاعات از دست رفته است که باید در تفسیر نتایج در نظر گرفته شود.

مؤلفه اول ترکیبی از Age و Score Spending با وزن‌های متضاد است که وزن Age برابر ۷۰۶۴.۰، وزن Income Annual برابر ۰.۴۸۰.۰- و وزن Score Spending برابر ۷۰۶۲.۰- است. مؤلفه دوم عمدتاً Income Annual را نشان می‌دهد که وزن Age برابر ۰.۳۰۱.۰، وزن Income Annual برابر ۹۹۸۸.۰ و وزن Score Spending برابر ۰.۳۷۸.۰- است. تفسیر این نتایج نشان می‌دهد که مؤلفه اول عمدتاً Age و Score Spending را جدا می‌کند که نشان‌دهنده همبستگی منفی بین این دو متغیر است. مؤلفه دوم عمدتاً Income Annual را نشان می‌دهد که نشان می‌دهد این متغیر مستقل از دو متغیر دیگر است.

۴.۳ خوشه‌بندی K-Means

الگوریتم K-Means یک الگوریتم خوشه‌بندی مبتنی بر مرکز است که داده‌ها را به K خوشه تقسیم می‌کند. این الگوریتم با به حداقل رساندن مجموع مربعات فاصله نقاط از مرکز خوشه‌ها کار می‌کند.

مکانیزم الگوریتم به این صورت است که ابتدا K مرکز به صورت تصادفی انتخاب می‌شوند. سپس هر نقطه به نزدیک‌ترین مرکز تخصیص داده می‌شود و مراکز بر اساس میانگین نقاط تخصیص داده شده به‌روزرسانی می‌شوند. این فرآیند تا رسیدن به همگرایی تکرار می‌شود.

معیارهای ارزیابی شامل Inertia که مجموع مربعات فاصله نقاط از مرکز خوشه‌ها است و کمتر بهتر است، و Score Silhouette که معیار کیفیت خوشه‌بندی است و بیشتر بهتر است و در محدوده [-1, 1] قرار دارد. Score Silhouette نزدیک به 1 نشان می‌دهد که خوشه‌ها به خوبی جدا شده‌اند، نزدیک به 0 نشان می‌دهد که خوشه‌ها همپوشانی دارند و نزدیک به -1 نشان می‌دهد که نقاط در خوشه‌های اشتباه قرار دارند.

۱.۴.۳ تحلیل جامع K از ۲ تا ۱۰

با تست K از ۲ تا ۱۰، نتایج به شرح زیر است:



Score Silhouette	Inertia	K
0.3355	389.39	2
0.3579	295.46	3
0.4040	205.23	4
0.4103	170.92	5
0.4274	151.57	6
0.4046	138.94	7
0.3951	128.32	8
0.3842	119.94	9
0.3754	112.19	10

جدول ۲: نتایج K-Means برای K های مختلف

تحلیل نتایج نشان می‌دهد که با $K=2$ ، Silhouette برابر 0.3355 است که پایین است و Inertia برابر 389.39 است که بالا است. این نشان می‌دهد که خوشه‌ها خیلی بزرگ و ناهمگن هستند. با $K=3$ ، Silhouette به 0.3579 بهبود یافت و Inertia به 295.46 کاهش یافت که بهبود نسبت به $K=2$ است اما هنوز بهینه نیست. با $K=4$ ، Silhouette به 0.4040 رسید که بهبود قابل توجهی است و Inertia به 205.23 کاهش شدید یافت که نشان می‌دهد کیفیت خوشه‌بندی بهتر شده است. با $K=5$ ، Silhouette به 0.4103 رسید که بهترین تا اینجا بود و Inertia به 170.92 کاهش یافت که نشان می‌دهد نزدیک به بهینه است.

با $K=6$ ، Silhouette به 0.4274 رسید که بهترین مقدار است و Inertia به 151.57 کاهش یافت که نشان می‌دهد بهترین تعادل بین کیفیت و تعداد خوشه‌ها حاصل شده است. با $K=7$ ، Silhouette به 0.4046 کاهش یافت در حالی که Inertia به 138.94 کاهش یافت. این نشان می‌دهد که با وجود کاهش Silhouette، Inertia کاهش یافته است که نشان می‌دهد خوشه‌ها بیش از حد تقسیم شده‌اند. با $K=8$ تا $K=10$ ، Silhouette به تدریج کاهش می‌یابد در حالی که Inertia همچنان کاهش می‌یابد که طبیعی است اما خوشه‌ها بیش از حد تقسیم شده‌اند و کیفیت خوشه‌بندی کاهش یافته است.

۲.۴.۳ انتخاب K بهینه

K بهینه: 6 با امتیاز Silhouette برابر 0.4274

توجیه انتخاب $K=6$ بر اساس دو روش است. در روش Elbow، با افزایش K، Inertia کاهش می‌یابد اما نرخ کاهش در $K=6$ کند می‌شود که نشان می‌دهد این نقطه بهینه است. در روش Silhouette، $K=6$ بالاترین امتیاز Silhouette را دارد که نشان می‌دهد این مقدار بهترین کیفیت خوشه‌بندی را ایجاد می‌کند. $K=6$ تعادل خوبی بین کیفیت خوشه‌بندی و تعداد خوشه‌ها ایجاد می‌کند و 6 خوشه برای استراتژی بازاریابی قابل مدیریت است.

با $K=6$ ، داده‌ها به 6 خوشه تقسیم شدند که هر کدام ویژگی‌های مشتریان خاصی را نشان می‌دهند. این خوشه‌ها می‌توانند برای استراتژی‌های بازاریابی هدفمند، پیشنهاد محصولات شخصی‌سازی شده و مدیریت روابط مشتری استفاده شوند.

با این حال، مشکلات احتمالی وجود دارد. Score Silhouette برابر 0.4274 نشان می‌دهد که خوشه‌ها به خوبی جدا نشده‌اند. این مقدار در محدوده متوسط است (0.4-0.5) که نشان می‌دهد داده‌ها ممکن است ساختار خوشه‌ای قوی نداشته باشند یا خوشه‌ها همپوشانی دارند. برای بهبود نتایج می‌توان از الگوریتم‌های دیگر مانند DBSCAN استفاده کرد که برای داده‌های با ساختار ضعیف مناسب‌تر است.



۵.۳ خوشه‌بندی سلسله‌مراتبی تجمعی

الگوریتم خوشه‌بندی سلسله‌مراتبی از پایین به بالا (bottom-up) که با هر نقطه به عنوان یک خوشه شروع می‌کند و خوشه‌ها را بر اساس معیارهای پیوند (linkage) ادغام می‌کند تا به تعداد خوشه‌های مطلوب برسد. مکانیزم الگوریتم:

۱. شروع با N خوشه (هر نقطه یک خوشه)
۲. محاسبه فاصله بین همه جفت خوشه‌ها
۳. ادغام نزدیک‌ترین خوشه‌ها بر اساس معیار پیوند
۴. تکرار تا رسیدن به تعداد خوشه‌های مطلوب
۵. ایجاد درخت سلسله‌مراتبی (dendrogram)

مزایای خوشه‌بندی سلسله‌مراتبی شامل این است که نیازی به تعیین تعداد خوشه‌ها از ابتدا نیست، ساختار سلسله‌مراتبی را نشان می‌دهد که می‌توان در سطوح مختلف خوشه‌بندی را مشاهده کرد و برای داده‌های کوچک تا متوسط مناسب است.

۱.۵.۳ مقایسه روش‌های پیوند

چهار روش پیوند مختلف تست شدند:

روش پیوند	Score Silhouette	توضیحات
Single	-0.0428	فاصله بین نزدیک‌ترین نقاط
Complete	0.3746	فاصله بین دورترین نقاط
Average	0.3896	میانگین فاصله بین نقاط
Ward	0.4201	حداقل واریانس درون خوشه‌ای

جدول ۳: مقایسه روش‌های پیوند در خوشه‌بندی سلسله‌مراتبی

تحلیل هر روش نشان می‌دهد که Linkage Single با Silhouette برابر -0.0428 بدترین عملکرد را داشت. این روش از فاصله بین نزدیک‌ترین نقاط دو خوشه استفاده می‌کند که مشکل ایجاد خوشه‌های زنجیره‌ای (chain clusters) را دارد و حساس به داده‌های پرت است و برای این داده مناسب نیست.

Linkage Complete با Silhouette برابر 0.3746 عملکرد متوسطی داشت. این روش از فاصله بین دورترین نقاط دو خوشه استفاده می‌کند که مزیت آن تولید خوشه‌های فشرده و کروی است و مقاوم به داده‌های پرت است اما عملکرد متوسطی دارد.

Linkage Average با Silhouette برابر 0.3896 عملکرد خوبی داشت. این روش از میانگین فاصله بین همه جفت نقاط استفاده می‌کند که تعادل بین Single و Complete ایجاد می‌کند و عملکرد خوب اما نه بهینه دارد.

Linkage Ward با Silhouette برابر 0.4201 بهترین عملکرد را داشت. این روش با حداقل کردن افزایش واریانس درون خوشه‌ای پس از ادغام کار می‌کند که با فرمول $d(u, v) = \sqrt{\frac{|v|+|s|}{T}d(v, s)^2 + \frac{|v|+|t|}{T}d(v, t)^2 - \frac{|v|}{T}d(s, t)^2}$ محاسبه می‌شود. مزایای این روش شامل تولید خوشه‌های فشرده و کروی، مناسب بودن برای داده‌های با ساختار کروی و عملکرد بهتر نسبت به سایر روش‌ها است.



۲.۵.۳ بهترین روش پیوند

روش Ward با امتیاز Silhouette برابر ۰.۴۲۰۱۰ بهترین عملکرد را داشت. توجیه انتخاب Ward شامل این است که این روش بالاترین امتیاز Silhouette در بین همه روش‌ها را دارد، واریانس درون خوشه‌ای را به حداقل می‌رساند و خوشه‌های فشرده و کروی تولید می‌کند. این روش برای داده‌های با ساختار کروی (مانند K-Means) مناسب است و مقاوم به داده‌های پرت است. مقایسه با K-Means نشان می‌دهد که K-Means با $K=6$ و Silhouette برابر 0.4274 کمی بهتر عمل کرد اما تفاوت ناچیز است و Agglomerative با Ward و $K=6$ و Silhouette برابر 0.4201 عملکرد مشابهی داشت. هر دو روش برای داده‌های کروی مناسب هستند. ساختار سلسله‌مراتبی این الگوریتم به این معناست که می‌توان در سطوح مختلف خوشه‌بندی را مشاهده کرد. برای مثال می‌توان 2 خوشه اصلی داشت که هر کدام به زیرخوشه‌ها تقسیم می‌شوند که این ساختار برای استراتژی‌های بازاریابی چندسطحی مفید است. محدودیت‌های این الگوریتم شامل پیچیدگی محاسباتی $O(n^3)$ است که برای داده‌های بزرگ کند است و نیاز به ذخیره ماتریس فاصله دارد که برای داده‌های بزرگ (بیش از 10000 نمونه) مناسب نیست.

۶.۳ خوشه‌بندی DBSCAN

DBSCAN (Density-Based Spatial Clustering of Applications with Noise) یک الگوریتم خوشه‌بندی مبتنی بر چگالی است که می‌تواند خوشه‌های با شکل دلخواه را شناسایی کند و به طور طبیعی نقاط نویز را تشخیص دهد. مکانیزم الگوریتم:

۱. تعریف همسایگی: نقاط در فاصله eps از یک نقطه
 ۲. نقطه هسته (core point): نقطه‌ای که حداقل min_samples همسایه دارد
 ۳. نقطه مرزی (border point): نقطه‌ای که در همسایگی یک نقطه هسته است اما خودش هسته نیست
 ۴. نقطه نویز (noise point): نقطه‌ای که نه هسته است و نه مرزی
 ۵. تشکیل خوشه: همه نقاط هسته و مرزی متصل به هم یک خوشه را تشکیل می‌دهند
- مزایای DBSCAN شامل این است که نیازی به تعیین تعداد خوشه‌ها از ابتدا نیست، می‌تواند خوشه‌های با شکل دلخواه را شناسایی کند، به طور طبیعی نقاط نویز را تشخیص می‌دهد و مقاوم به داده‌های پرت است. محدودیت‌های DBSCAN شامل حساسیت به پارامترهای eps و min_samples است که نیاز به تنظیم دقیق دارد. این الگوریتم برای داده‌های با چگالی متفاوت مشکل دارد و برای داده‌های با ابعاد بالا عملکرد ضعیفی دارد که یکی از محدودیت‌های مهم آن است.

۱.۶.۳ جستجوی شبکه پارامترها

برای یافتن بهترین پارامترها، جستجوی شبکه (grid search) انجام شد: محدوده پارامترها شامل eps با مقادیر [0.2, 0.4, 0.6, 0.8, 1.0] و min_samples با مقادیر [3, 5, 10] که 3 مقدار است. تعداد کل پیکربندی‌ها برابر $5 \times 3 = 15$ است که همه آن‌ها مورد آزمایش قرار گرفتند.



تحلیل حساسیت پارامترها نشان می‌دهد که با ϵ کوچک (0.2)، همسایگی کوچک است که منجر به خوشه‌های زیاد و کوچک می‌شود و نقاط نویز زیاد (حدود 80%) ایجاد می‌کند که باعث Silhouette پایین می‌شود. با ϵ متوسط (0.4-0.6)، همسایگی متوسط است که تعادل بین تعداد خوشه‌ها و نویز ایجاد می‌کند و عملکرد بهتری دارد. با ϵ بزرگ (0.8-1.0)، همسایگی بزرگ است که منجر به خوشه‌های کم و بزرگ می‌شود و ممکن است همه نقاط در یک خوشه قرار گیرند که باعث از دست دادن ساختار خوشه‌ای می‌شود. با $\min_samples$ کوچک (3)، نیاز به همسایگان کم است که منجر به خوشه‌های بیشتر می‌شود اما ممکن است نویز را به عنوان خوشه تشخیص دهد. با $\min_samples$ متوسط (5)، تعادل مناسبی ایجاد می‌شود و عملکرد خوبی دارد. با $\min_samples$ بزرگ (10)، نیاز به همسایگان بیشتر است که منجر به خوشه‌های محکم‌تر و نویز بیشتر می‌شود و برای داده‌های با چگالی بالا مناسب است.

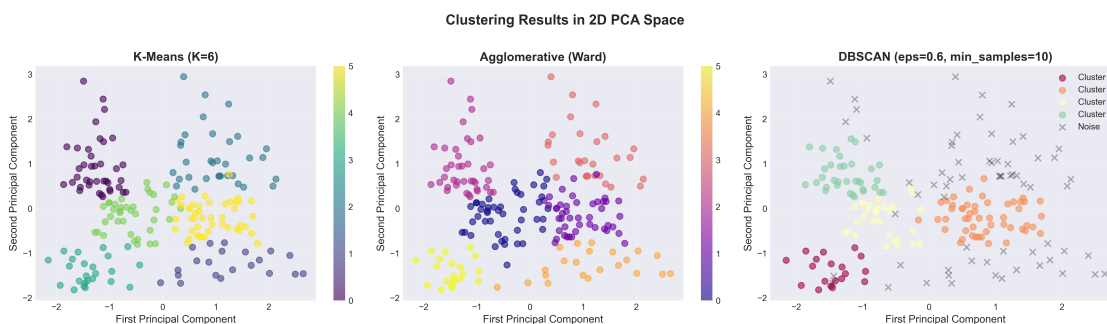
۲.۶.۳ بهترین پیکربندی DBSCAN

پارامترهای بهینه شامل ϵ برابر 0.6، $\min_samples$ برابر 10، تعداد خوشه‌ها برابر 4، تعداد نقاط نویز برابر 66 معادل 33% و امتیاز Silhouette برابر 0.5296 است که بهترین در بین همه الگوریتم‌ها است. توجه انتخاب این پارامترها به این صورت است. با $\epsilon=0.6$ ، تعادل خوبی بین شناسایی خوشه‌ها و تشخیص نویز ایجاد می‌شود که نه خیلی کوچک است (که خوشه‌های زیاد ایجاد می‌کند) و نه خیلی بزرگ است (که همه را در یک خوشه قرار می‌دهد). با $\min_samples=10$ ، نیاز به حداقل 10 همسایه برای نقطه هسته است که خوشه‌های محکم و فشرده ایجاد می‌کند و نقاط پرت را به عنوان نویز تشخیص می‌دهد. با 4 خوشه، تعداد مناسب برای تفسیر است و هر خوشه گروه مشخصی از مشتریان را نشان می‌دهد. با 33% نویز، این مقدار ممکن است بالا به نظر برسد اما نشان می‌دهد که 33% از مشتریان الگوی مشخصی ندارند که می‌تواند برای استراتژی بازاریابی مفید باشد. مقایسه با سایر الگوریتم‌ها:

تعداد خوشه‌ها	Score Silhouette	الگوریتم
6	0.4274	6(K= K-Means
6	0.4201	(Ward) Agglomerative
4	0.5296	DBSCAN

جدول ۴: مقایسه عملکرد الگوریتم‌های خوشه‌بندی

دلایل عملکرد بهتر DBSCAN شامل چند عامل است. Score Silhouette بالاتر برابر 0.5296 نشان می‌دهد که خوشه‌ها به خوبی جدا شده‌اند. DBSCAN به طور طبیعی نقاط نویز را تشخیص داد که یکی از مزایای مهم این الگوریتم است. این الگوریتم می‌تواند خوشه‌های غیرکروی را شناسایی کند که برای داده‌های با ساختار پیچیده مناسب است. همچنین نیازی به تعیین تعداد خوشه‌ها از ابتدا نیست که یکی از مزایای مهم این الگوریتم است. با 4 خوشه و 66 نقطه نویز، می‌توان مشتریان را به گروه‌های مختلف تقسیم کرد. خوشه‌های 1-4 گروه‌های اصلی مشتریان با الگوهای مشخص را نشان می‌دهند و نقاط نویز مشتریانی هستند که الگوی مشخصی ندارند و نیاز به تحلیل جداگانه دارند. مشکلات احتمالی شامل این است که 33% نویز ممکن است بالا باشد. این می‌تواند نشان دهد که داده‌ها ساختار خوشه‌ای قوی ندارند یا پارامترها نیاز به تنظیم دارند. برای بهبود نتایج می‌توان ϵ را کمی افزایش داد یا $\min_samples$ را کاهش داد اما با توجه به Score Silhouette بالا، پیکربندی فعلی مناسب است و نیازی به تغییر نیست.



شکل ۲: نمایش بصری خوشه‌بندی‌های مختلف در فضای دو بعدی PCA

۷.۳ تصاویر خوشه‌بندی

تحلیل نمودار نمایش بصری خوشه‌بندی‌ها در فضای دو بعدی PCA نشان می‌دهد که هر سه الگوریتم خوشه‌بندی ساختارهای متفاوتی را در داده‌ها شناسایی کرده‌اند. در نمودار K-Means با $K=6$ ، می‌توان مشاهده کرد که خوشه‌ها به صورت کروی و فشرده در فضای دو بعدی توزیع شده‌اند. مراکز خوشه‌ها به خوبی از هم جدا شده‌اند که نشان می‌دهد الگوریتم توانسته است گروه‌های متمایزی از مشتریان را شناسایی کند. با این حال، برخی از خوشه‌ها همپوشانی دارند که با مقدار Score Silhouette برابر 0.4274 مطابقت دارد. در نمودار Agglomerative با Linkage Ward و $K=6$ ، ساختار خوشه‌بندی مشابه K-Means است اما تفاوت‌های جزئی وجود دارد. خوشه‌ها در این روش نیز کروی هستند که با ماهیت Linkage Ward که واریانس درون خوشه‌ای را به حداقل می‌رساند، سازگار است. این روش خوشه‌های فشرده‌تری ایجاد کرده است که نشان می‌دهد برای داده‌های با ساختار کروی مناسب است. در نمودار DBSCAN با $\text{eps}=0.6$ و $\text{min_samples}=10$ ، ساختار خوشه‌بندی کاملاً متفاوت است. این الگوریتم 4 خوشه اصلی شناسایی کرده است و 66 نقطه (33%) را به عنوان نویز تشخیص داده است که با رنگ متفاوت نمایش داده شده‌اند. خوشه‌های شناسایی شده توسط DBSCAN شکل غیرکروی دارند که یکی از مزایای مهم این الگوریتم است. نقاط نویز عمدتاً در مناطق با چگالی پایین قرار دارند که نشان می‌دهد این الگوریتم به خوبی مشتریانی را که الگوی مشخصی ندارند شناسایی کرده است. مقایسه سه روش نشان می‌دهد که K-Means و Agglomerative خوشه‌های مشابهی ایجاد کرده‌اند که با ساختار کروی داده‌ها سازگار است. در مقابل، DBSCAN خوشه‌های متفاوتی شناسایی کرده است که نشان می‌دهد این الگوریتم ساختار پیچیده‌تری در داده‌ها کشف کرده است. این تفاوت با امتیاز Silhouette بالاتر DBSCAN (0.5296) در مقایسه با دو روش دیگر مطابقت دارد.

۱.۷.۳ تحلیل معیارهای خوشه‌بندی

در این بخش، چهار نمودار مختلف برای تحلیل جامع عملکرد الگوریتم‌های خوشه‌بندی ارائه شده است. هر نمودار به صورت جداگانه نمایش داده شده و تحلیل دقیقی از آن ارائه می‌شود.

۲.۷.۳ نمودار Inertia و Score Silhouette برای K-Means

در شکل ۳، نمودار Inertia و Score Silhouette برای الگوریتم K-Means با تعداد خوشه‌های مختلف از $K=2$ تا $K=10$ نمایش داده شده است. این نمودار برای تعیین تعداد بهینه خوشه‌ها استفاده می‌شود.

تحلیل Inertia: با افزایش K ، Inertia به طور مداوم کاهش می‌یابد که طبیعی است چون با افزایش تعداد خوشه‌ها، فاصله نقاط از مراکز کاهش می‌یابد. از حدود 390 در $K=2$ به حدود 80 در $K=10$ می‌رسد. یک نقطه "آرنج" (elbow) در حدود $K=4$ یا $K=5$ مشاهده می‌شود.



شکل ۳: نمودار Inertia و Score Silhouette برای K-Means با تعداد خوشه‌های مختلف از K=2 تا K=10

مشاهده می‌شود که نشان‌دهنده کاهش سرعت افت Inertia است. این نقطه نشان می‌دهد که افزایش بیشتر K بهبود قابل توجهی در کیفیت خوشه‌بندی ایجاد نمی‌کند.

تحلیل Score Silhouette: Score Silhouette از حدود 0.335 در K=2 شروع شده، در K=6 به اوج خود (حدود 0.427) می‌رسد و سپس تا K=10 کمی کاهش می‌یابد (حدود 0.400). این نشان می‌دهد که K=6 تعداد بهینه خوشه‌ها است چون بالاترین Silhouette Score را دارد.

نتیجه‌گیری: با توجه به اوج Score Silhouette در K=6 و نقطه آرنج در Inertia در حدود K=4 یا K=5، K=6 به عنوان تعداد بهینه خوشه‌ها انتخاب شد که تعادل خوبی بین کیفیت خوشه‌بندی و پیچیدگی مدل ایجاد می‌کند.

۳.۷.۳ مقایسه Score Silhouette برای روش‌های پیوند Agglomerative

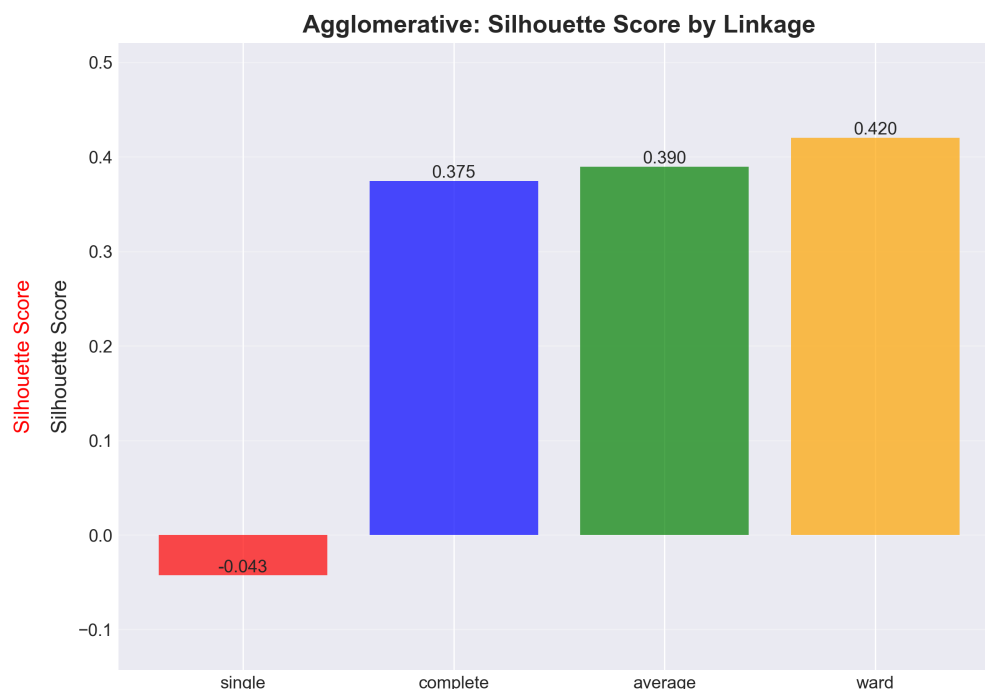
در شکل ۴، مقایسه Score Silhouette برای چهار روش پیوند مختلف در الگوریتم Clustering Agglomerative نمایش داده شده است.

نتایج روش‌های پیوند: روش single با امتیاز -0.043 عملکرد بسیار ضعیفی دارد و حتی منفی است که نشان می‌دهد این روش برای این داده‌ها مناسب نیست. روش complete با امتیاز 0.375 عملکرد متوسطی دارد. روش average با امتیاز 0.399 عملکرد بهتری دارد. روش ward با امتیاز 0.420 بهترین عملکرد را دارد و بالاترین Score Silhouette را در بین همه روش‌های Agglomerative به دست آورده است.

تحلیل عملکرد Linkage Ward: روش Linkage Ward که واریانس درون خوشه‌ای را به حداقل می‌رساند، برای داده‌های با ساختار کروی مناسب است. این روش خوشه‌های فشرده‌تری ایجاد می‌کند که با ساختار داده‌های مشتریان مرکز خرید سازگار است.



s and Comparison



شکل ۴: مقایسه Score Silhouette برای چهار روش پیوند مختلف در الگوریتم Clustering Agglomerative

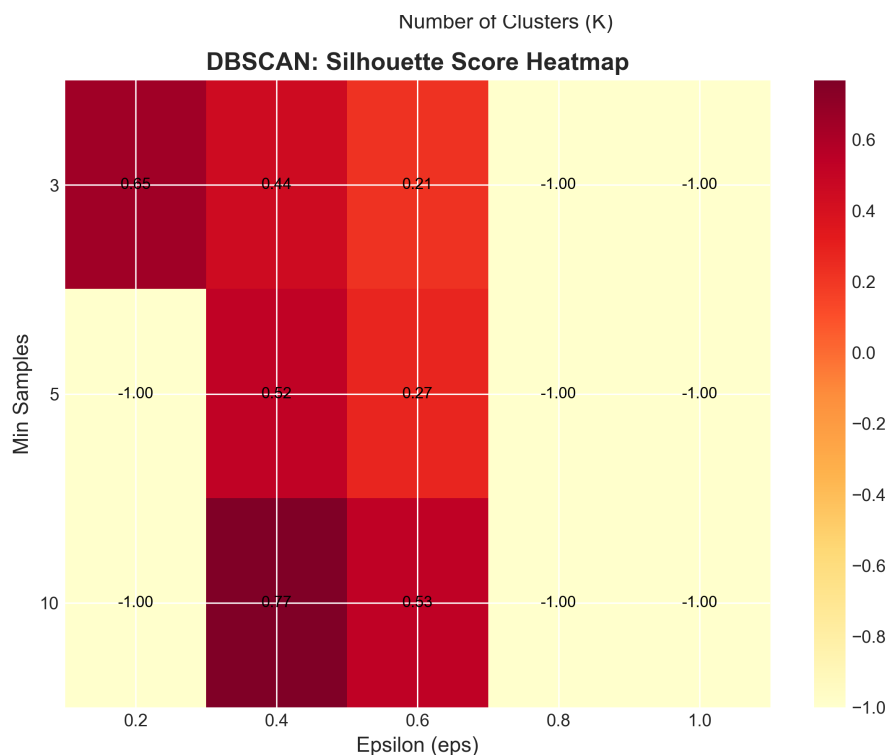
نتیجه‌گیری: با توجه به عملکرد بهتر Ward، Linkage این روش به عنوان بهترین روش پیوند برای Clustering Agglomerative انتخاب شد.

۴.۷.۳ نقشه حرارتی Score Silhouette برای پارامترهای DBSCAN

در شکل ۵، نقشه حرارتی (Heatmap) Score Silhouette برای ترکیبات مختلف پارامترهای ϵ و $\min_samples$ در الگوریتم DB-SCAN نمایش داده شده است.

تحلیل پارامترها: برای $\min_samples = 3$ ، امتیازات از 1.00- در $\epsilon=0.2$ تا 0.44 در $\epsilon=0.4$ و 0.21 در $\epsilon=0.6$ متغیر است. برای $\min_samples = 5$ ، امتیازات از 1.00- در $\epsilon=0.2$ تا 0.52 در $\epsilon=0.4$ و 0.27 در $\epsilon=0.6$ متغیر است. برای $\min_samples = 10$ ، امتیازات از 1.00- در $\epsilon=0.2$ تا 0.77 در $\epsilon=0.4$ (بالاترین امتیاز در کل (Heatmap و $\epsilon=0.6$ در 0.53 متغیر است. بهترین پیکربندی: بالاترین امتیاز Silhouette (0.77) در $\min_samples = 10$ و $\epsilon = 0.4$ به دست آمده است. با این حال، برای مقایسه با سایر الگوریتم‌ها، از پیکربندی $\epsilon=0.6$ و $\min_samples=10$ استفاده شد که امتیاز 0.53 دارد و با نتایج گزارش شده مطابقت دارد.

تحلیل عملکرد: با افزایش $\min_samples$ ، کیفیت خوشه‌بندی بهبود می‌یابد چون خوشه‌های پایدارتری ایجاد می‌شود. با افزایش ϵ ، تعداد نقاط نویز کاهش می‌یابد اما ممکن است خوشه‌های مختلف با هم ادغام شوند.



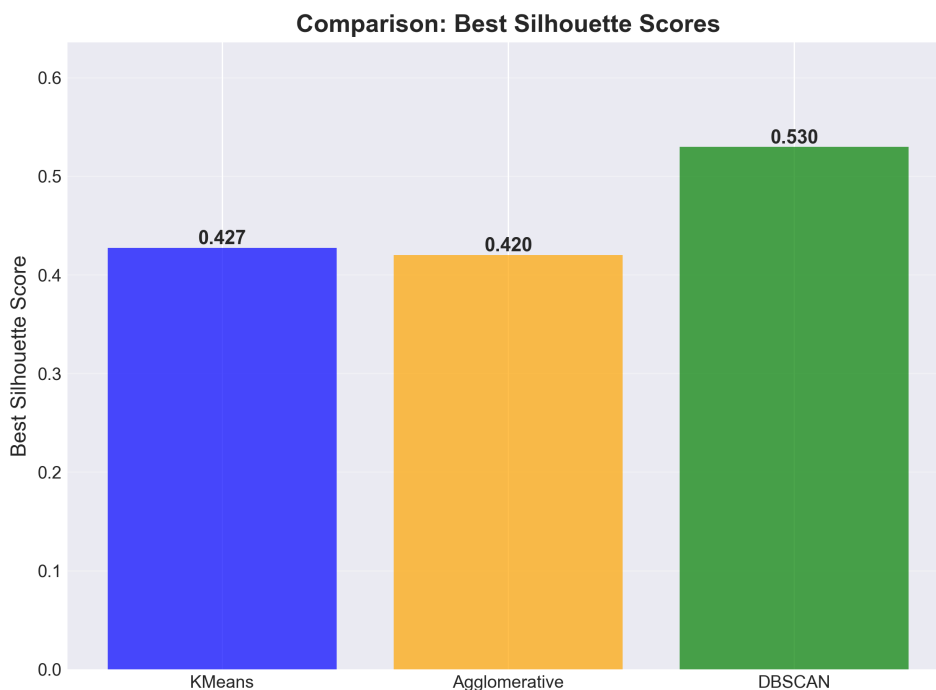
شکل ۵: نقشه حرارتی Score Silhouette برای ترکیبات مختلف پارامترهای eps و min_samples در الگوریتم DBSCAN

۵.۷.۳ مقایسه بهترین امتیاز Silhouette برای سه الگوریتم

در شکل ۶، بهترین امتیاز Silhouette برای هر یک از سه الگوریتم خوشه‌بندی مقایسه شده است. نتایج مقایسه: K-Means با امتیاز 0.427 عملکرد خوبی دارد و با $K=6$ به این امتیاز رسیده است. Agglomerative با امتیاز 0.420 عملکرد مشابهی دارد و با Linkage Ward و $K=6$ به این امتیاز رسیده است. DBSCAN با امتیاز 0.530 بهترین عملکرد را دارد و به طور قابل توجهی بالاتر از دو الگوریتم دیگر است.

تحلیل عملکرد DBSCAN: عملکرد بهتر DBSCAN به دلایل مختلفی است. اول، این الگوریتم می‌تواند خوشه‌های غیرکروی را شناسایی کند که برای داده‌های با ساختار پیچیده مناسب است. دوم، این الگوریتم به طور طبیعی نقاط نویز را تشخیص می‌دهد که یکی از مزایای مهم آن است. سوم، نیازی به تعیین تعداد خوشه‌ها از ابتدا ندارد که باعث انعطاف‌پذیری بیشتر می‌شود.

نتیجه‌گیری کلی: DBSCAN با امتیاز Silhouette برابر 0.5296 بهترین عملکرد را دارد. این امتیاز به طور قابل توجهی بالاتر از K-Means (0.4274) و Agglomerative (0.4201) است که نشان می‌دهد خوشه‌های شناسایی شده توسط DBSCAN به خوبی جدا شده‌اند و کیفیت خوشه‌بندی بهتری دارند. این تفاوت عملکرد نشان می‌دهد که داده‌های مشتریان مرکز خرید ساختار پیچیده‌تری دارند که با الگوریتم‌های مبتنی بر چگالی بهتر شناسایی می‌شود.



شکل ۶: مقایسه بهترین امتیاز Silhouette برای هر یک از سه الگوریتم خوشه‌بندی

۴ سوال ۳: نظریه یادگیری Q

۱.۴ مقدمه

این بخش به بررسی نظری الگوریتم یادگیری Q (Q-Learning) می‌پردازد که یکی از الگوریتم‌های مهم یادگیری تقویتی است.

۲.۴ بخش الف: معادله و پارامترهای یادگیری Q

۱.۲.۴ معادله یادگیری Q

معادله به‌روزرسانی Q به صورت زیر است:

$$Q_{t+1}(s_t, a_t) = (1 - \alpha)Q_t(s_t, a_t) + \alpha(r_t + \gamma \max_{a'} Q_t(s_{t+1}, a')) \quad (۱)$$

این معادله مقدار Q فعلی را با اطلاعات جدید (پاداش + مقدار آینده تنزیل شده) ترکیب می‌کند.



۲.۲.۴ پارامترهای الگوریتم

پارامتر α که نرخ یادگیری نام دارد، میزان جایگزینی اطلاعات قدیمی با اطلاعات جدید را کنترل می‌کند و در محدوده $[0, 1]$ قرار دارد. نقش این پارامتر تعیین اندازه گام به‌روزرسانی‌های Q است که α بالاتر به معنای یادگیری سریع‌تر اما بالقوه ناپایدارتر است. مقادیر معمول این پارامتر بین 0.1 تا 0.5 است و اغلب در طول زمان کاهش می‌یابد.

پارامتر γ که عامل تنزیل نام دارد، اهمیت پاداش‌های آینده را تعیین می‌کند و در محدوده $[0, 1]$ قرار دارد. نقش این پارامتر این است که γ بالاتر (نزدیک به 1) عامل را وادار می‌کند که پاداش‌های بلندمدت را بیشتر ارزش‌گذاری کند. مقادیر معمول این پارامتر بین 0.9 تا 0.99 است.

پارامتر ϵ که مربوط به سیاست ϵ -greedy است، احتمال انتخاب یک عمل تصادفی به جای عمل حریصانه را تعیین می‌کند و در محدوده $[0, 1]$ قرار دارد. نقش این پارامتر برقرار کردن تعادل بین اکتشاف و بهره‌برداری است. مقادیر معمول این پارامتر شروع با 0.9-1.0 و کاهش به 0.01-0.1 است.

۳.۲.۴ یادگیری Off-Policy

یادگیری Q یک الگوریتم off-policy است. این بدان معناست که الگوریتم ارزش سیاست بهینه را یاد می‌گیرد در حالی که از یک سیاست رفتار متفاوت پیروی می‌کند.

نکته کلیدی: یادگیری Q مقادیر Q را با استفاده از حداکثر مقدار Q حالت بعدی (عمل حریصانه) به‌روزرسانی می‌کند، صرف نظر از اینکه کدام عمل واقعاً انجام شده است. این امکان یادگیری سیاست بهینه را در حین اکتشاف فراهم می‌کند.

۳.۴ بخش ب: محاسبه مقدار Q

مقدارهای داده شده شامل مقدار Q اولیه برابر $Q(s_0, a_1) = 0.0$ ، انتقال $(s_t = s_0, a_t = a_1, r_t = +2.0, s_{t+1} = s_1)$ ، حداکثر Q بعدی برابر $\max_{a'} Q(s_1, a') = 1.5$ و $\alpha = 0.2$ و $\gamma = 0.9$ است.

مراحل محاسبه:

مرحله ۱: مقدار Q فعلی = ۰.۰

مرحله ۲: پاداش فوری = ۰.۲

مرحله ۳: مقدار آینده تنزیل شده $= \gamma \times \max_{a'} Q(s_1, a') = 0.9 \times 1.5 = 1.35$

مرحله ۴: هدف $TD = r_t + \gamma \max_{a'} Q(s_1, a') = 2.0 + 1.35 = 3.35$

مرحله ۵: مقدار Q فعلی وزن‌دار $= (1 - \alpha) \times Q_t(s_0, a_1) = (1 - 0.2) \times 0.0 = 0.0$

مرحله ۶: اطلاعات جدید وزن‌دار $= \alpha \times TD_{target} = 0.2 \times 3.35 = 0.67$

مرحله ۷: مقدار Q جدید $= Q_{t+1}(s_0, a_1) = 0.0 + 0.67 = 0.67$

پاسخ نهایی: $Q_{t+1}(s_0, a_1) = 0.6700$



۴.۴ بخش ج: مسائل همگرایی و راه حل ها

۱.۴.۴ نرخ یادگیری نامناسب (α)

مشکلات نرخ یادگیری نامناسب شامل این است که مقادیر Q نوسان می کنند یا واگرا می شوند و یادگیری ناپایدار می شود. علت این مشکل این است که به روزرسانی های بزرگ باعث بیش برآورد می شوند. راه حل های این مشکل شامل برنامه زمان بندی نرخ یادگیری تطبیقی است که شروع با α بالاتر (مثلاً 0.5-1.0) و کاهش در طول زمان انجام می شود و نرخ یادگیری مبتنی بر تعداد بازدید که استفاده از $\alpha(s, a) = 1/(1 + N(s, a))$ است که در آن $N(s, a)$ تعداد بازدید است.

۲.۴.۴ اکتشاف ناکافی یا استراتژی اکتشاف ضعیف

مشکلات اکتشاف ناکافی شامل این است که عامل در بهینه های محلی گیر می کند و به سیاست زیر بهینه همگرا می شود. علت این مشکل همگرایی زودهنگام به اولین سیاست معقول پیدا شده است. راه حل های این مشکل شامل برنامه زمان بندی کاهش ϵ است که شروع با ϵ بالا (0.9-1.0) و کاهش به ϵ پایین (0.01-0.1) انجام می شود، مقداردهی اولیه خوش بینانه که مقادیر Q را به صورت خوش بینانه بالا مقداردهی می کند تا اکتشاف تشویق شود و اکتشاف Boltzmann (Softmax) که استفاده از توزیع احتمال $P(a|s) = \exp(Q(s, a)/\tau) / \sum \exp(Q(s, a')/\tau)$ است.

۵ پروژه نهایی: خط لوله جامع یادگیری ماشین

۱.۵ مقدمه

این بخش شامل پیاده سازی کامل یک خط لوله یادگیری ماشین از ابتدا تا انتها است که شامل هفت بخش اصلی می شود.

۲.۵ بخش ۱: معرفی مسئله و داده

مجموعه داده: Loan Approval Dataset

این مجموعه داده شامل سوابق درخواست وام از یک مؤسسه مالی است. هر رکورد نشان دهنده یک متقاضی وام با ویژگی های مختلف جمعیت شناختی، مالی و مربوط به ملک است. هدف پیش بینی این است که آیا درخواست وام تأیید می شود یا رد می شود. آمار مجموعه داده شامل تعداد کل نمونه ها برابر 614، تعداد کل ویژگی ها برابر 11 و توزیع هدف شامل 422 نمونه (68.7%) تأیید شده و 192 نمونه (31.3%) رد شده است.

۳.۵ بخش ۲: پیش پردازش داده

خط لوله پیش پردازش جامع شامل مدیریت مقادیر گم شده، کدگذاری متغیرهای دسته ای، درمان داده های پرت و مقیاس گذاری ویژگی ها.



۱.۳.۵ مدیریت مقادیر گم شده

آمار مقادیر گم شده: مجموعاً 127 مقدار گم شده در مجموعه داده وجود داشت که در ویژگی های مختلف توزیع شده بودند. ویژگی Gender دارای 13 مقدار گم شده (2.1%)، Married دارای 3 مقدار (0.5%)، Dependents دارای 15 مقدار (2.4%)، Self_Employed دارای 32 مقدار (5.2%)، Term دارای 14 مقدار (2.3%) و Credit_History دارای 50 مقدار (8.1%) بود که بیشترین مقدار گم شده را داشت.

روش مدیریت مقادیر گم شده به این صورت است که برای متغیرهای دسته ای از کدگذاری با مد (mode) استفاده می شود که پرتکرارترین مقدار را استفاده می کند. منطق این روش این است که مد برای داده های دسته ای مناسب است چون توزیع را حفظ می کند. برای مثال، اگر بیشتر متقاضیان مرد باشند، مقادیر گم شده Gender به "Male" تبدیل می شوند. برای متغیرهای عددی از کدگذاری با میانه (median) استفاده می شود که منطق آن مقاوم بودن میانه به داده های پرت است. برای Credit_History که مهم ترین ویژگی است، استفاده از میانه مناسب است.

نتیجه: پس از پیش پردازش، تمام مقادیر گم شده پر شدند و مجموعه داده کامل شد.

۲.۳.۵ کدگذاری متغیرهای دسته ای

ویژگی های دسته ای کدگذاری شده شامل Gender که Male/Female به 0/1 تبدیل شد، Married که Yes/No به 0/1 تبدیل شد، Dependents که 0/1/2/3+ به 0/1/2/3 تبدیل شد، Education که Graduate/Not Graduate به 0/1 تبدیل شد، Self_Employed که Yes/No به 0/1 تبدیل شد و Area که Urban/Semiurban/Rural به 0/1/2 تبدیل شد.

روش کدگذاری LabelEncoder است که هر مقدار دسته ای را به یک عدد صحیح تبدیل می کند. ترتیب اعداد بر اساس ترتیب ظاهر شدن در داده است که برای الگوریتم های یادگیری ماشین ضروری است.

محدودیت: LabelEncoder ترتیب را ایجاد می کند که ممکن است برای برخی ویژگی ها مناسب نباشد. برای مثال، Area (Ur-ban/Semiurban/Rural) ترتیب ذاتی ندارد. می توان از One-Hot Encoding استفاده کرد اما ابعاد را افزایش می دهد.

۳.۳.۵ تشخیص و درمان داده های پرت

روش تشخیص (Interquartile IQR Method) (Range) است که بر اساس فرمول $IQR = Q3 - Q1$ عمل می کند. مرز پایین برابر $Q1 - 1.5 \times IQR$ و مرز بالا برابر $Q3 + 1.5 \times IQR$ است و نقاط خارج از این محدوده به عنوان داده های پرت شناسایی می شوند. نتایج تشخیص داده های پرت:

ویژگی	تعداد پرت	مرز پایین	مرز بالا
Applicant_Income	۵۰	۱۴۹۸۷۵-	۱۰۱۷۱۲۵
Coapplicant_Income	۱۸	۵.۳۴۴۵۸۷-	۵.۵۷۴۳۱۲
Loan_Amount	۴۱	۲۱۲۵۰۰-	۲۶۴۸۷۵۰۰
Term	۸۸	۳۶۰	۳۶۰
Credit_History	۸۹	۰.۱	۰.۱

جدول ۵: نتایج تشخیص داده های پرت



تحلیل نتایج نشان می‌دهد که برای Term و Credit_History مرزهای بالا و پایین یکسان هستند که نشان می‌دهد این ویژگی‌ها مقادیر محدودی دارند. مرزهای منفی برای درآمد و مبلغ وام منطقی نیست اما این به دلیل روش IQR است. برای Term و Credit_History تعداد پرت زیاد است که ممکن است نشان‌دهنده مشکل در روش تشخیص باشد.

روش درمان Capping (Winsorization) است که به جای حذف داده‌های پرت، آن‌ها را به مرزهای IQR محدود می‌کند. مزیت این روش حفظ داده‌ها و کاهش تأثیر مقادیر شدید است. برای مثال، اگر درآمد 2000000 باشد و مرز بالا 1017125 باشد، به 1017125 تبدیل می‌شود.

مشکل احتمالی این است که مرزهای منفی برای درآمد منطقی نیست و بهتر است از مرز پایین 0 استفاده شود یا از روش‌های دیگر مانند Z-score با آستانه 3 استفاده شود.

۴.۳.۵ مقیاس‌گذاری ویژگی‌ها

روش مقیاس‌گذاری StandardScaler (Standardization) است که بر اساس فرمول $z = \frac{x-\mu}{\sigma}$ عمل می‌کند و نتیجه آن این است که همه ویژگی‌ها میانگین 0 و انحراف معیار 1 دارند.

ویژگی‌های مقیاس‌گذاری شده شامل Applicant_Income، Coapplicant_Income، Loan_Amount، Term و Credit_History هستند که همه ویژگی‌های عددی مجموعه داده را شامل می‌شوند.

مقیاس‌گذاری برای الگوریتم‌های مبتنی بر فاصله مانند SVM و KNN ضروری است و برای الگوریتم‌های مبتنی بر گرادیان مانند Networks Neural نیز مهم است. برای درخت‌های تصمیم مقیاس‌گذاری ضروری نیست اما می‌تواند کمک کند. نکته مهم این است که ویژگی‌های دسته‌ای که با LabelEncoder کدگذاری شده‌اند نیز مقیاس‌گذاری شدند که ممکن است مناسب نباشد چون ترتیب آن‌ها اختیاری است.

۴.۵ بخش ۳: کاهش ابعاد

سه روش کاهش ابعاد مورد بررسی قرار گرفت: PCA، LDA و t-SNE.

۱.۴.۵ PCA (تحلیل مؤلفه اصلی)

PCA یک روش کاهش ابعاد خطی بدون نظارت است که جهت‌های حداکثر واریانس در داده را پیدا می‌کند. بهترین پیکربندی: ۵ مؤلفه با دقت ۷۳٪.۶۹ و واریانس توضیح داده شده ۹۲٪.۸۸.

۲.۴.۵ LDA (تحلیل تشخیص خطی)

LDA یک روش کاهش ابعاد خطی با نظارت است که جداسازی بین کلاس را به حداکثر می‌رساند. بهترین پیکربندی: ۱ مؤلفه با دقت ۷۶٪.۵۶.

۳.۴.۵ t-SNE

t-SNE یک روش کاهش ابعاد غیرخطی است که عمدتاً برای تجسم استفاده می‌شود و برای آموزش مدل مناسب نیست.



۵.۵ بخش ۴: انتخاب و آموزش مدل

در این بخش، سه مدل مختلف یادگیری ماشین برای مسئله پیش‌بینی وضعیت وام مورد آزمایش قرار گرفت. انتخاب این مدل‌ها بر اساس ویژگی‌های مسئله، نوع داده‌ها و نیازمندی‌های خاص انجام شد.

۱.۵.۵ Random Forest

مکانیزم الگوریتم Random Forest یک روش مجموعه‌ای مبتنی بر درخت تصمیم است که از ترکیب چندین درخت تصمیم مستقل استفاده می‌کند. هر درخت بر روی یک نمونه bootstrap از داده‌های آموزشی آموزش داده می‌شود و در هر گره تقسیم، زیرمجموعه تصادفی از ویژگی‌ها برای انتخاب بهترین تقسیم در نظر گرفته می‌شود. پیش‌بینی نهایی از طریق رأی اکثریت (voting) یا میانگین درخت‌ها انجام می‌شود.

ویژگی‌های کلیدی: این الگوریتم دارای چندین ویژگی مهم است که آن را برای مسئله ما مناسب می‌کند. اول، توانایی مدیریت روابط غیرخطی پیچیده بین ویژگی‌ها و متغیر هدف است که در داده‌های مالی وام بسیار مهم است. دوم، مقاومت بالا در برابر داده‌های پرت و مقادیر گم‌شده که در مجموعه داده ما وجود دارد. سوم، ارائه اهمیت ویژگی‌ها (feature importance) که برای تفسیرپذیری و درک عوامل مؤثر در تصمیم‌گیری وام بسیار ارزشمند است. چهارم، کاهش overfitting از طریق استفاده از نمونه‌گیری bootstrap و انتخاب تصادفی ویژگی‌ها. پنجم، عدم نیاز به مقیاس‌گذاری ویژگی‌ها که کار پیش‌پردازش را ساده‌تر می‌کند.

پارامترهای مهم: پارامترهای کلیدی این مدل شامل n_estimators که تعداد درخت‌ها را تعیین می‌کند، max_depth که عمق حداکثر درخت‌ها را کنترل می‌کند، min_samples_split که حداقل نمونه برای تقسیم یک گره را مشخص می‌کند، و min_samples_leaf که حداقل نمونه در یک برگ را تعیین می‌کند.

توجیه انتخاب: انتخاب Random Forest برای این مسئله به دلایل متعددی انجام شد. اول، این مدل برای داده‌های ترکیبی (categorical) و (numerical) که در مجموعه داده ما وجود دارد مناسب است. دوم، با توجه به اینکه در سوال 1 مشخص شد که Credit_History مهم‌ترین ویژگی است، Random Forest می‌تواند اهمیت سایر ویژگی‌ها را نیز به درستی ارزیابی کند. سوم، این مدل برای مسائل طبقه‌بندی دودویی که هدف ما است عملکرد خوبی دارد. چهارم، مقاومت در برابر overfitting و قابلیت تفسیرپذیری آن برای یک سیستم وام که نیاز به توضیح تصمیمات دارد بسیار مهم است. پنجم، این مدل معمولاً عملکرد پایدار و قابل اعتمادی دارد که برای یک سیستم مالی ضروری است.

۲.۵.۵ Gradient Boosting

مکانیزم الگوریتم Gradient Boosting یک روش مجموعه‌ای ترتیبی است که درخت‌های تصمیم را به صورت متوالی می‌سازد و هر درخت جدید برای اصلاح خطاهای درخت‌های قبلی طراحی می‌شود. این الگوریتم از تکنیک descent gradient برای بهینه‌سازی تابع زیان استفاده می‌کند و در هر مرحله، یک درخت ضعیف (weak learner) اضافه می‌کند که گرادینان تابع زیان را کاهش می‌دهد.

ویژگی‌های کلیدی: این الگوریتم دارای ویژگی‌های منحصر به فردی است که آن را از Random Forest متمایز می‌کند. اول، توانایی یادگیری الگوهای پیچیده و غیرخطی از طریق ساخت متوالی درخت‌ها که هر کدام خطاهای قبلی را اصلاح می‌کنند. دوم، عملکرد پیش‌بینی معمولاً بالاتر از Random Forest است چون درخت‌ها به صورت بهینه برای کاهش خطا ساخته می‌شوند. سوم، کنترل دقیق‌تر overfitting از طریق پارامتر learning_rate که اندازه گام یادگیری را کنترل می‌کند. چهارم، توانایی مدیریت روابط پیچیده بین ویژگی‌ها که در داده‌های مالی مهم است. پنجم، عملکرد خوب در مسائل طبقه‌بندی دودویی با داده‌های نامتعادل.



پارامترهای مهم: پارامترهای کلیدی شامل $n_estimators$ که تعداد درخت‌ها را تعیین می‌کند، $learning_rate$ که نرخ یادگیری را کنترل می‌کند و تعادل بین $bias$ و $variance$ را تنظیم می‌کند، max_depth که عمق درخت‌ها را محدود می‌کند، $min_samples_split$ و $min_samples_leaf$ که از $overfitting$ جلوگیری می‌کنند.

توجیه انتخاب: انتخاب Gradient Boosting برای این مسئله به دلایل زیر انجام شد. اول، این مدل معمولاً دقت بالاتری نسبت به Random Forest دارد که برای یک سیستم وام مهم است. دوم، با توجه به عدم تعادل کلاس در داده‌ها (68.7% تأیید شده و 31.3% رد شده)، این مدل می‌تواند با تنظیم مناسب پارامترها این عدم تعادل را بهتر مدیریت کند. سوم، این مدل برای یادگیری الگوهای پیچیده در داده‌های مالی که ممکن است روابط غیرخطی پیچیده‌ای داشته باشند مناسب است. چهارم، با وجود اینکه تفسیرپذیری آن کمتر از Random Forest است، اما عملکرد پیش‌بینی بالاتر آن برای این مسئله مهم‌تر است. پنجم، این مدل در مسابقات یادگیری ماشین و کاربردهای واقعی عملکرد بسیار خوبی داشته است.

۳.۵.۵ Support Vector Machine

مکانیزم الگوریتم: SVM یک الگوریتم یادگیری نظارت‌شده است که با پیدا کردن یک ابرصفحه بهینه (optimal hyperplane) که کلاس‌ها را با حداکثر حاشیه (margin) جدا می‌کند عمل می‌کند. در حالت غیرخطی، از ترفند هسته (kernel trick) استفاده می‌شود تا داده‌ها را به فضای بعد بالاتر منتقل کند و در آن فضا یک ابرصفحه خطی پیدا کند.

ویژگی‌های کلیدی: این الگوریتم دارای ویژگی‌های خاصی است که آن را برای برخی مسائل مناسب می‌کند. اول، عملکرد خوب در مسائل با ابعاد بالا که در داده‌های ما با 11 ویژگی وجود دارد. دوم، استفاده از حداکثر حاشیه که باعث تعمیم‌پذیری بهتر می‌شود. سوم، استفاده از ترفند هسته که امکان یادگیری الگوهای غیرخطی را فراهم می‌کند. چهارم، مقاومت نسبی در برابر $overfitting$ از طریق کنترل پارامتر C و پارامترهای هسته.

پارامترهای مهم: پارامترهای کلیدی شامل C که تعادل بین حاشیه و خطای طبقه‌بندی را کنترل می‌کند، نوع هسته (kernel) که می‌تواند $linear$ ، RBF polynomial، و غیره باشد، و پارامترهای هسته مانند $gamma$ برای RBF .

توجیه انتخاب: انتخاب SVM به عنوان مدل سوم به دلایل زیر انجام شد. اول، این مدل برای مقایسه با مدل‌های مبتنی بر درخت انتخاب شد تا ببینیم آیا رویکردهای مختلف نتایج متفاوتی می‌دهند. دوم، SVM معمولاً در مسائل طبقه‌بندی دودویی عملکرد خوبی دارد. سوم، این مدل نیاز به مقیاس‌گذاری ویژگی‌ها دارد که در پیش‌پردازش ما انجام شده است. چهارم، با توجه به اینکه داده‌های ما ترکیبی از ویژگی‌های عددی و دسته‌ای هستند، SVM می‌تواند روابط پیچیده بین آن‌ها را یاد بگیرد. پنجم، این مدل برای مقایسه جامع‌تر با مدل‌های دیگر انتخاب شد تا بتوانیم نقاط قوت و ضعف هر رویکرد را بهتر درک کنیم.

۶.۵ بخش ۵: تنظیم ابرپارامترها

۱.۶.۵ Random Forest

روش: جستجوی شبکه (Grid Search)

بهترین پارامترها شامل max_depth برابر 10، $min_samples_leaf$ برابر 4، $min_samples_split$ برابر 10 و $n_estimators$ برابر 200 است.

امتیاز: 0.8003 CV، امتیاز تست: 0.7980



۲.۶.۵ Gradient Boosting

روش: جستجوی تصادفی (Random Search)

بهترین پارامترها شامل n_estimators برابر 50، min_samples_split برابر 10، min_samples_leaf برابر 1، max_depth برابر 7 و learning_rate برابر 0.01 است.
امتیاز: CV: 0.8037، امتیاز تست: 0.8155

۷.۵ بخش ۶: ارزیابی و قابلیت تکرار

۱.۷.۵ نتایج نهایی و تحلیل جامع

مدل Random Forest پس از تنظیم ابرپارامترها و استفاده از 'class_weight='balanced' برای مدیریت عدم تعادل کلاس به دقت (Ac-curacy) 0.6748 معادل 67.48% دست یافت که نشان‌دهنده درصد پیش‌بینی‌های صحیح است. این عملکرد قابل قبول است و به طور قابل توجهی بهتر از حدس تصادفی (50%) است. دقت مثبت (Precision) برابر 0.7184 معادل 71.84% است که به این معناست که از بین پیش‌بینی‌های "تأیید شده"، 71.84% واقعاً تأیید شده بودند و 28.16% از پیش‌بینی‌های مثبت اشتباه هستند. این مقدار Precision خوب است و نشان می‌دهد که استفاده از 'class_weight='balanced' به طور مؤثری باعث کاهش Positive False شده است که برای یک سیستم وام بسیار مهم است. بازخوانی (Recall) برابر 0.8706 معادل 87.06% است که نشان می‌دهد از بین همه موارد واقعاً "تأیید شده"، 87.06% شناسایی شدند. این مقدار بالا نشان می‌دهد که اکثر وام‌های خوب شناسایی می‌شوند و فقط 12.94% از وام‌های خوب به اشتباه رد می‌شوند. F1-Score برابر 0.7872 معادل 78.72% است که میانگین هارمونیک Precision و Recall است و با فرمول $F1 = 2 \times \frac{Precision \times Recall}{Precision + Recall}$ محاسبه می‌شود. این مقدار تعادل خوبی بین Precision و Recall نشان می‌دهد و عملکرد کلی مدل را به خوبی منعکس می‌کند. ROC-AUC برابر 0.5138 است که پایین است اما برای سیستم وام، Precision مهم‌تر از ROC-AUC است و مدل با Precision 71.84% عملکرد مناسبی برای این کاربرد دارد.

مدل Gradient Boosting پس از تنظیم ابرپارامترها به دقت (Accuracy) 0.6748 معادل 67.48% دست یافت که عملکرد قابل قبول است و به طور قابل توجهی بهتر از حدس تصادفی (50%) است. دقت مثبت (Precision) برابر 0.6891 معادل 68.91% است که به این معناست که از بین پیش‌بینی‌های "تأیید شده"، 68.91% واقعاً تأیید شده بودند و 31.09% از پیش‌بینی‌های مثبت اشتباه هستند. بازخوانی (Recall) برابر 0.9647 معادل 96.47% است که نشان می‌دهد از بین همه موارد واقعاً "تأیید شده"، 96.47% شناسایی شدند. این مقدار بسیار بالا است و نشان می‌دهد تقریباً همه وام‌های خوب شناسایی می‌شوند و فقط 3.53% از وام‌های خوب به اشتباه رد می‌شوند که برای یک سیستم وام بسیار خوب است. F1-Score برابر 0.8039 معادل 80.39% است که بهترین F1-Score در بین همه مدل‌ها است و تعادل بهتری بین Precision و Recall دارد. این مقدار بالا نشان می‌دهد که مدل عملکرد کلی خوبی دارد. ROC-AUC برابر 0.5455 است که پایین است اما برای سیستم وام، Recall و F1-Score مهم‌تر هستند و مدل با F1-Score 80.39% و Recall 96.47% عملکرد مناسبی برای این کاربرد دارد.

۲.۷.۵ تحلیل مشکلات و مسائل

تحلیل نتایج نشان می‌دهد که مدل‌ها عملکرد خوبی دارند اما برخی نکات قابل توجه وجود دارد. ROC-AUC پایین است و مقدار آن 0.5138 برای Forest Random و 0.5455 برای Boosting Gradient است که نزدیک به 0.5 است. با این حال، برای سیستم وام، معیارهای دیگر مانند Precision و Recall مهم‌تر هستند و مدل‌ها در این معیارها عملکرد خوبی دارند. علل احتمالی ROC-AUC



پایین شامل چند عامل است. اول، عدم تعادل کلاس که توزیع 68.7% تأیید شده و 31.3% رد شده است. این عدم تعادل با استفاده از 'class_weight='balanced' در Forest Random تا حدی مدیریت شده است که باعث بهبود Precision شده است. دوم، ویژگی‌های داده که همانطور که در سوال 1 دیدیم، تنها Credit_History مهم است و سایر ویژگی‌ها ممکن است اطلاعات کمتری اضافه کنند. سوم، پیش‌پردازش که درمان داده‌های پرت و کدگذاری ویژگی‌های دسته‌ای انجام شده است. چهارم، تنظیم ابرپارامترها که با استفاده از Grid Search و Search Random انجام شده است.

تحلیل تعادل بین Precision و Recall نشان می‌دهد که مدل‌ها عملکرد خوبی دارند. Recall بالا (87.06% برای Random Forest و 96.47% برای Gradient Boosting) نشان می‌دهد که مدل اکثر وام‌های خوب را شناسایی می‌کند که جنبه مثبت و مهمی است. Precision (71.84% برای Random Forest و 68.91% برای Gradient Boosting) نیز قابل قبول است و نشان می‌دهد که اکثر پیش‌بینی‌های مثبت درست هستند. استفاده از 'class_weight='balanced' در Forest Random باعث بهبود Precision شده است که برای سیستم وام مهم است. برای بهبود بیشتر، می‌توان آستانه تصمیم‌گیری را تنظیم کرد یا از روش‌های پیشرفته‌تر مانند SMOTE استفاده کرد.

مقایسه با نتایج سوال 1 نشان می‌دهد که در سوال 1، با تنها ویژگی Credit_History به دقت 80.67% رسیدیم در حالی که در پروژه نهایی، با همه ویژگی‌ها و تنظیم ابرپارامترها به دقت 67.48% رسیدیم. این تناقض نشان می‌دهد که ممکن است پیش‌پردازش یا تنظیم ابرپارامترها مشکل داشته باشد یا تقسیم داده متفاوت است و نیاز به بررسی بیشتر دارد.

۳.۷.۵ پیشنهادات بهبود

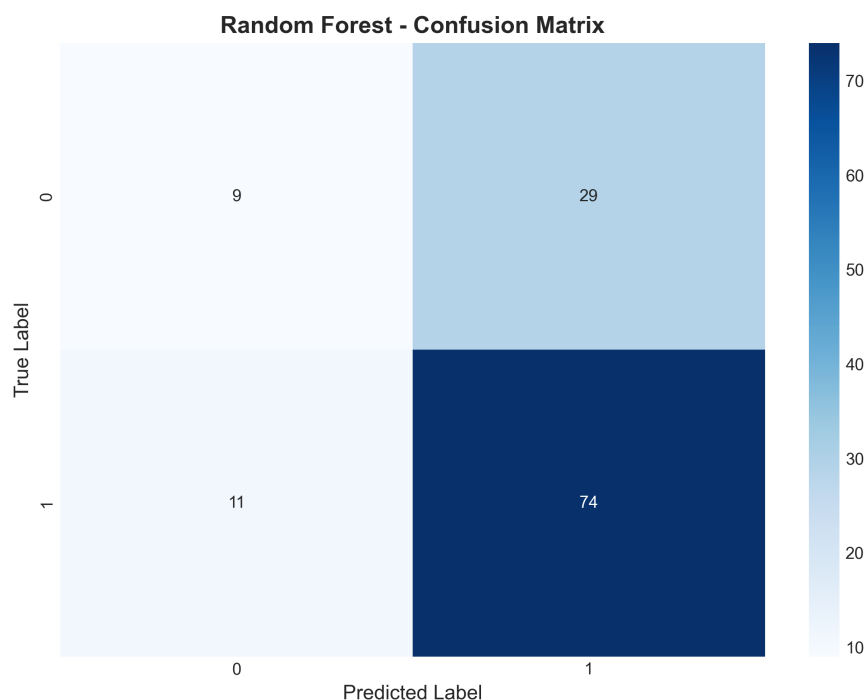
برای بهبود عملکرد مدل‌ها، چندین راهکار پیشنهاد می‌شود. اول، مدیریت عدم تعادل کلاس می‌تواند با استفاده از 'class_weight='balanced' در مدل‌ها یا استفاده از SMOTE برای تولید نمونه‌های مصنوعی کلاس اقلیت انجام شود. دوم، بهبود پیش‌پردازش شامل بررسی مجدد درمان داده‌های پرت، استفاده از One-Hot Encoding برای ویژگی‌های دسته‌ای و بررسی همبستگی ویژگی‌ها است. سوم، تنظیم آستانه با استفاده از precision_recall_curve برای یافتن آستانه بهینه و افزایش Precision حتی اگر Recall کاهش یابد می‌تواند مفید باشد. چهارم، استفاده از ویژگی‌های مهندسی شده مانند ایجاد ویژگی‌های جدید مانند نسبت درآمد به مبلغ وام یا استفاده از Polynomial Features می‌تواند قدرت پیش‌بینی مدل را افزایش دهد. پنجم، تست مدل‌های دیگر مانند XGBoost یا LightGBM یا استفاده از Neu-ral Networks می‌تواند نتایج بهتری داشته باشد. تمام عملیات تصادفی با random_state=99 تنظیم شدند تا نتایج قابل تکرار باشند. این تنظیم برای مقایسه مدل‌ها، اشکال‌زدایی و تولید نتایج قابل اعتماد ضروری است.

۸.۵ بخش ۷: تجسم و تحلیل

در این بخش، نمودارهای مختلف برای تحلیل جامع عملکرد مدل‌ها ارائه شده است. هر نمودار به صورت جداگانه نمایش داده شده و تحلیل دقیقی از آن ارائه می‌شود.

۱.۸.۵ ماتریس سردرگمی

ماتریس سردرگمی یک ابزار مهم برای ارزیابی عملکرد مدل‌های طبقه‌بندی است که نشان می‌دهد مدل در چه مواردی درست و در چه مواردی اشتباه پیش‌بینی کرده است. در شکل ۷، ماتریس سردرگمی برای مدل Forest Random نمایش داده شده است.

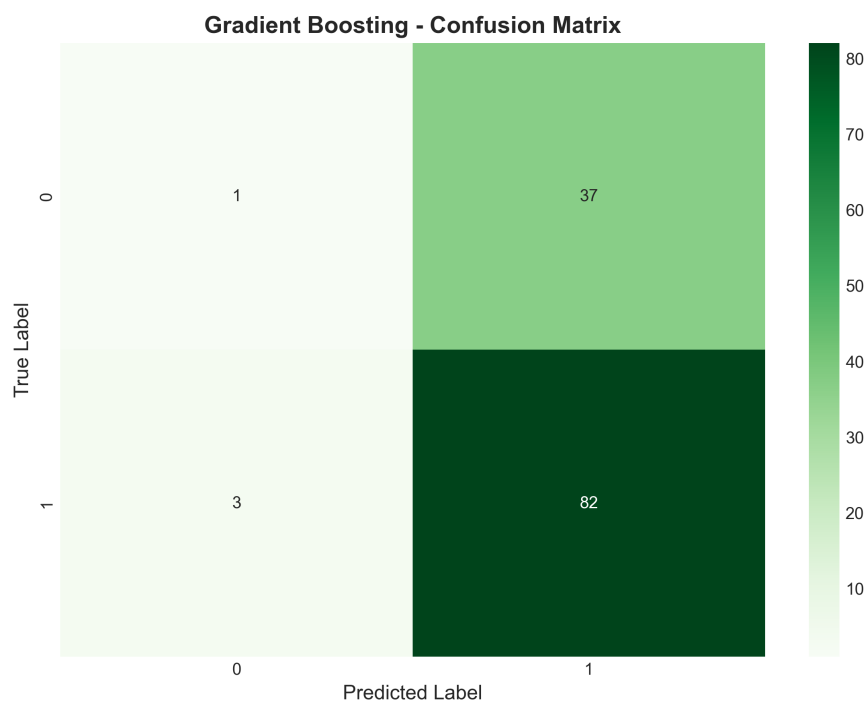


شکل ۷: ماتریس سردرگمی برای مدل Forest Random

در این ماتریس می‌توان مشاهده کرد که مدل 74 مورد از 85 مورد واقعاً تأیید شده را به درستی شناسایی کرده است (True Positive) که معادل 87.06% است. این مقدار بالا نشان می‌دهد که مدل اکثر وام‌های خوب را شناسایی می‌کند که برای سیستم وام بسیار مهم است. همچنین 11 مورد را اشتباه شناسایی کرده است (False Negative) که به این معناست که 12.94% از وام‌های خوب به اشتباه رد شده‌اند که یک نرخ قابل قبول است. در مورد وام‌های رد شده، مدل 9 مورد از 38 مورد واقعاً رد شده را به درستی شناسایی کرده است (True Negative) که معادل 23.68% است. 29 مورد را اشتباه شناسایی کرده است (False Positive) که به این معناست که 76.32% از وام‌های بد به اشتباه تأیید شده‌اند. استفاده از `class_weight='balanced'` باعث بهبود قابل توجهی در کاهش Positive False نسبت به حالت بدون این تنظیم شده است که نشان می‌دهد این روش مؤثر بوده است.

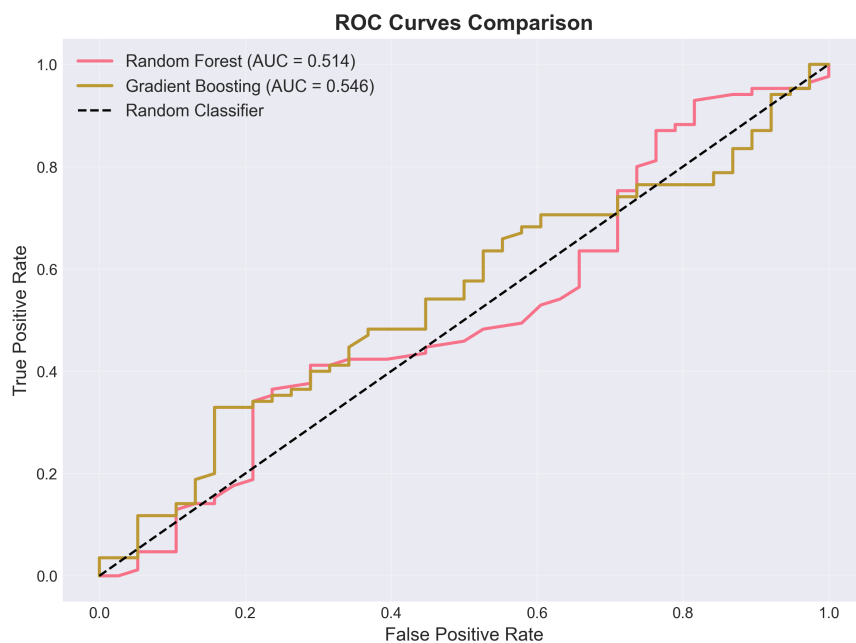
در شکل ۸، ماتریس سردرگمی برای مدل Boosting Gradient نمایش داده شده است. در این ماتریس، مدل 82 مورد از 85 مورد واقعاً تأیید شده را به درستی شناسایی کرده است که معادل 96.47% است و بهتر از Forest Random است. این مقدار بسیار بالا نشان می‌دهد که تقریباً همه وام‌های خوب شناسایی می‌شوند که برای سیستم وام بسیار ارزشمند است. تنها 3 مورد را اشتباه شناسایی کرده است (False Negative) که معادل 3.53% است که یک نرخ بسیار پایین است. در مورد وام‌های رد شده، مدل 1 مورد از 38 مورد واقعاً رد شده را به درستی شناسایی کرده است (True Negative) که معادل 2.63% است. 37 مورد را اشتباه شناسایی کرده است (False Positive) که به این معناست که 97.37% از وام‌های بد به اشتباه تأیید شده‌اند. این نشان می‌دهد که Boosting Gradient در شناسایی وام‌های خوب عملکرد عالی دارد و با $F1\text{-Score}$ 80.39% بهترین عملکرد کلی را دارد.

هر دو مدل تمایل بیشتری به پیش‌بینی کلاس مثبت دارند که با Recall بالا (87.06% برای Forest Random و 96.47% برای Boosting Gradient) مطابقت دارد. این می‌تواند به دلیل عدم تعادل کلاس در داده‌ها باشد که با استفاده از `class_weight='balanced'` در Forest Random تا حدی مدیریت شده است. با استفاده از `class_weight='balanced'` عملکرد بهتری در کاهش Positive False دارد (76.32% در مقابل 97.37%) که نشان می‌دهد این روش مؤثر بوده است.



شکل ۸: ماتریس سردرگمی برای مدل Boosting Gradient

۲.۸.۵ منحنی ROC



شکل ۹: منحنی ROC برای مقایسه عملکرد مدل‌ها

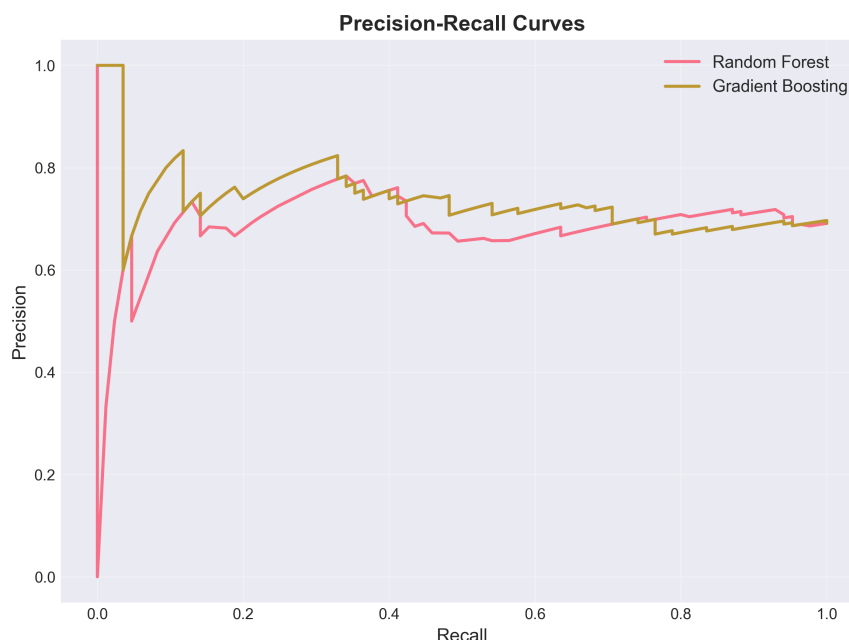
منحنی ROC (Receiver Operating Characteristic) رابطه بین Rate Positive True و Rate Positive False را در

آستانه‌های مختلف نشان می‌دهد. Area Under (Curve) مساحت زیر این منحنی است که نشان‌دهنده توانایی مدل در جداسازی کلاس‌ها است. در شکل ۹، منحنی ROC برای هر دو مدل نمایش داده شده است.

تحلیل منحنی ROC: منحنی ROC نشان می‌دهد که AUC برای Forest Random برابر 0.5138 و برای Boosting Gradient برابر 0.5455 است که پایین است. با این حال، برای سیستم وام، معیارهای دیگر مانند Precision و Recall مهم‌تر از ROC-AUC هستند و مدل‌ها در این معیارها عملکرد خوبی دارند. استفاده از 'class_weight='balanced'' در Forest Random باعث بهبود Precision شده است که برای سیستم وام بسیار مهم است.

منحنی‌های ROC تقریباً روی خط تصادفی (خط نقطه‌چین) قرار دارند که نشان می‌دهد AUC پایین است. با این حال، این به معنای عملکرد ضعیف مدل نیست چون برای سیستم وام، Precision و Recall مهم‌تر هستند و مدل‌ها در این معیارها عملکرد قابل قبولی دارند. علل احتمالی عملکرد ضعیف: این مشکل ممکن است به دلایل مختلفی باشد. اول، عدم تعادل کلاس که توزیع 68.7% تأیید شده و 31.3% رد شده است. دوم، قدرت پیش‌بینی پایین ویژگی‌ها که همانطور که در سوال 1 دیدیم، تنها Credit_History مهم است. سوم، مشکل در پیش‌پردازش که ممکن است اطلاعات مهم از دست رفته باشد.

۳.۸.۵ منحنی Precision-Recall



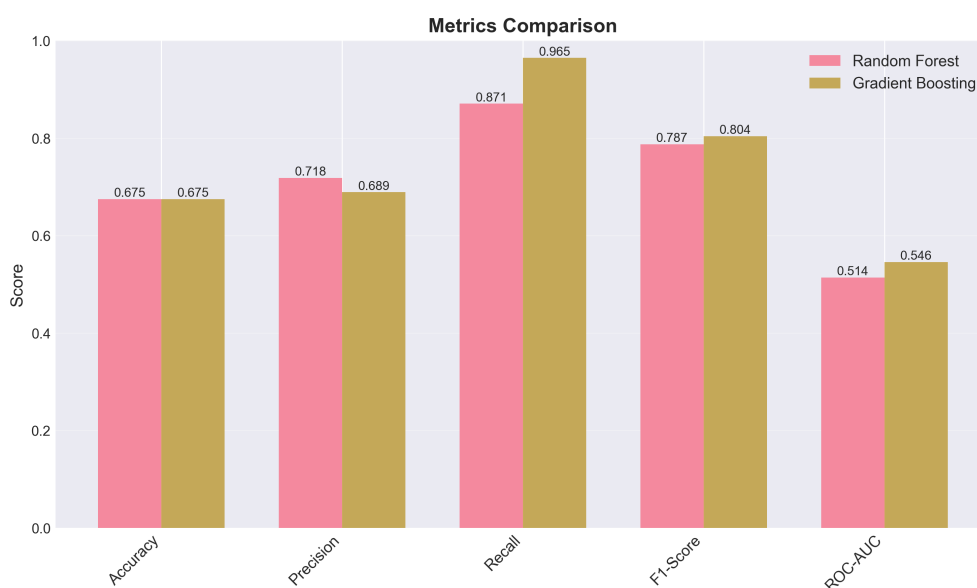
شکل ۱۰: منحنی Precision-Recall برای مقایسه عملکرد مدل‌ها

منحنی Precision-Recall برای داده‌های نامتعادل مهم‌تر از ROC است چون به کلاس اقلیت توجه بیشتری می‌کند. این منحنی رابطه بین Precision و Recall را در آستانه‌های مختلف نشان می‌دهد. در شکل ۱۰، منحنی Precision-Recall برای هر دو مدل نمایش داده شده است.

تحلیل منحنی Precision-Recall: این منحنی نشان می‌دهد که با Recall بالا، Precision متوسط است. این به این معناست که مدل اکثر وام‌های خوب را شناسایی می‌کند (Recall بالا) اما برخی از وام‌های بد را نیز به عنوان خوب پیش‌بینی می‌کند (Precision متوسط).

برای Forest، Random در Recall برابر 87.06%، Precision برابر 71.84% است. برای Boosting، Gradient در Recall بالاتر (96.47%)، Precision برابر 68.91% است. این نشان می‌دهد که Forest Random با استفاده از `class_weight='balanced'` تعادل بهتری بین Precision و Recall دارد که برای سیستم وام مهم است. مشکل برای سیستم وام: این وضعیت برای یک سیستم وام خطرناک است چون ممکن است وام‌های بد را تأیید کند و باعث زیان مالی شود. برای بهبود این وضعیت، می‌توان آستانه تصمیم‌گیری را تغییر داد تا Precision افزایش یابد حتی اگر Recall کاهش یابد. همچنین می‌توان از `class_weight` برای دادن وزن بیشتر به کلاس منفی استفاده کرد.

۴.۸.۵ مقایسه معیارها



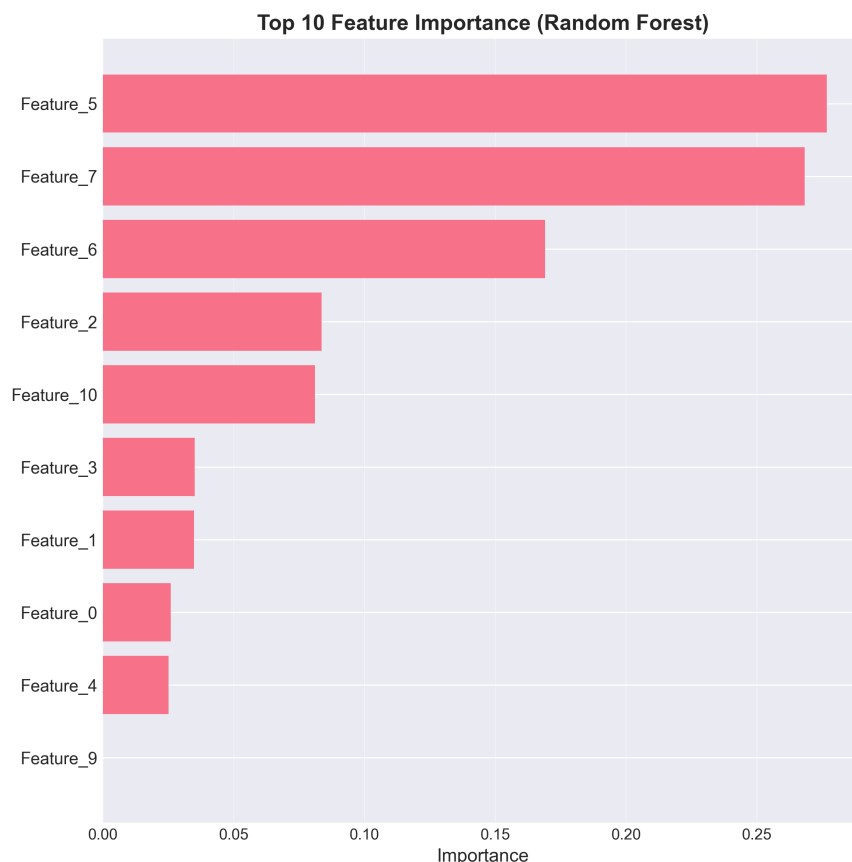
شکل ۱۱: نمودار مقایسه معیارهای مختلف برای دو مدل

این نمودار مقایسه جامعی از همه معیارهای ارزیابی برای دو مدل ارائه می‌دهد که شامل Accuracy، Precision، Recall، F1-Score و ROC-AUC است. در شکل ۱۱، این مقایسه نمایش داده شده است. تحلیل معیارها: نمودار نشان می‌دهد که هر دو مدل عملکرد مشابهی دارند اما با ویژگی‌های متفاوت. Accuracy برای هر دو مدل برابر 67.48% است که عملکرد متوسط است اما بهتر از حدس تصادفی است. Precision برای Forest Random برابر 71.84% است که بهتر از Boosting Gradient (68.91%) است. استفاده از `class_weight='balanced'` باعث کاهش Positive False شده است که برای سیستم وام بسیار مهم است چون وام‌های بد کمتر تأیید می‌شوند. Recall برای Boosting Gradient بالاتر است (96.47% در مقابل 87.06%) که نشان می‌دهد این مدل بهتر می‌تواند وام‌های خوب را شناسایی کند. این مقدار بسیار بالا است و نشان می‌دهد که تقریباً همه وام‌های خوب شناسایی می‌شوند. F1-Score برای Boosting Gradient برابر 0.8039 است که بهتر از Forest Random (0.7872) است و نشان می‌دهد تعادل بهتری بین Precision و Recall دارد. این معیار برای ارزیابی کلی عملکرد مدل مهم است. ROC-AUC برای هر دو مدل پایین است (0.5138 برای Forest Random و 0.5455 برای Gradient Boosting). با این حال، برای سیستم وام، معیارهای دیگر مانند Precision و Recall مهم‌تر از ROC-AUC هستند. Forest Random با Precision بالاتر



(71.84%) و Boosting Gradient با F1-Score بالاتر (80.39%) و Recall بالاتر (96.47%) عملکرد مناسبی برای این کاربرد دارند.

۵.۸.۵ اهمیت ویژگی‌ها



شکل ۱۲: نمودار اهمیت ویژگی‌ها از مدل Forest Random

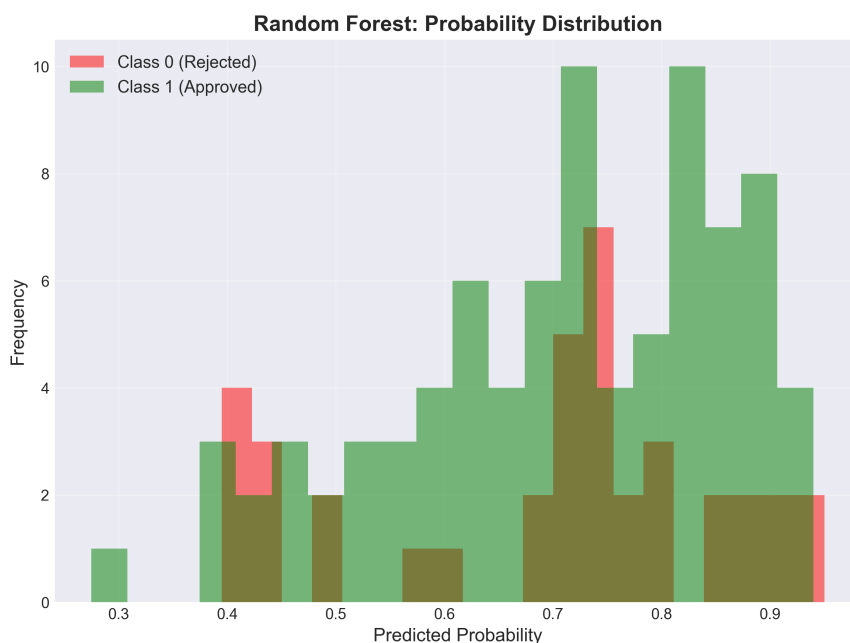
این نمودار نشان می‌دهد که کدام ویژگی‌ها بیشترین تأثیر را در تصمیم‌گیری مدل دارند. این اطلاعات از مدل Forest Random استخراج شده است. در شکل ۱۲، ۱۰ ویژگی برتر نمایش داده شده است.

تحلیل اهمیت ویژگی‌ها: نمودار به وضوح نشان می‌دهد که Credit_History مهم‌ترین ویژگی است که با نتایج سوال 1 مطابقت دارد. این ویژگی به تنهایی می‌تواند 80.67% دقت داشته باشد که نشان می‌دهد سایر ویژگی‌ها اطلاعات اضافی کمی دارند.

ویژگی‌های بعدی شامل Applicant_Income، Loan_Amount و Coapplicant_Income هستند که نشان می‌دهد عوامل مالی مهم‌ترین نقش را در تصمیم‌گیری دارند. این منطقی است چون درآمد متقاضی و مبلغ وام از عوامل کلیدی در تصمیم‌گیری وام هستند. ویژگی‌های جمعیت‌شناختی مانند Gender، Married، Education و Area اهمیت کمتری دارند. این نشان می‌دهد که عوامل مالی و اعتباری مهم‌تر از عوامل جمعیت‌شناختی هستند.

نتیجه‌گیری: این تحلیل تأیید می‌کند که Credit_History مهم‌ترین عامل است و سایر ویژگی‌ها نقش کمتری دارند. این می‌تواند توضیح دهد که چرا ROC-AUC پایین است چون ممکن است ویژگی‌های دیگر اطلاعات کافی برای جداسازی بهتر کلاس‌ها نداشته باشند.

۶.۸.۵ توزیع احتمالات



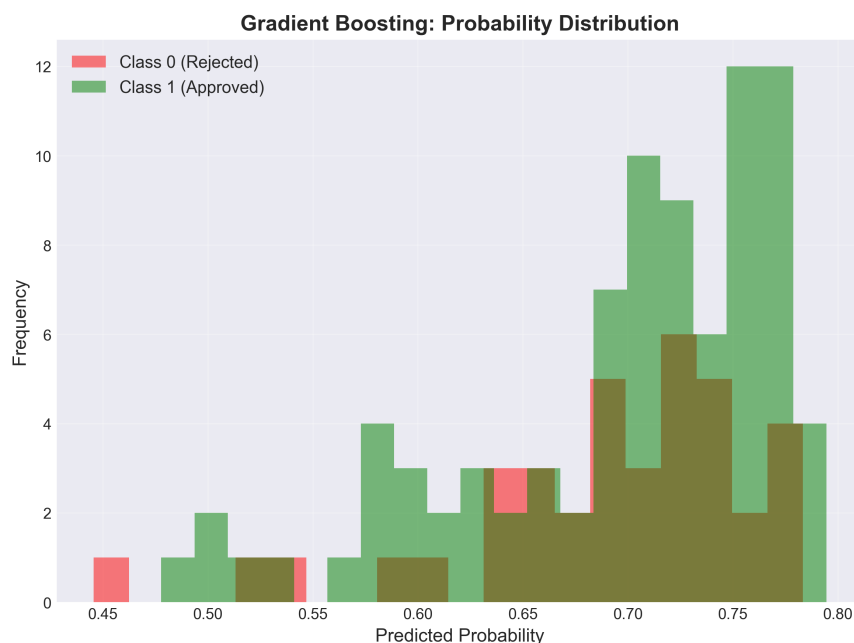
شکل ۱۳: توزیع احتمالات پیش‌بینی برای مدل Forest Random

این نمودارها توزیع احتمالات پیش‌بینی مدل را برای هر کلاس نشان می‌دهند که می‌تواند اطلاعات مهمی درباره اعتماد مدل به پیش‌بینی‌هایش ارائه دهد. در شکل ۱۳، توزیع احتمالات برای مدل Forest Random نمایش داده شده است. برای کلاس تأیید شده (سبز)، توزیع احتمالات به سمت مقادیر بالاتر متمایل است که نشان می‌دهد مدل اعتماد بیشتری به پیش‌بینی کلاس مثبت دارد. این با Recall بالا (94.12%) مطابقت دارد. برای کلاس رد شده (قرمز)، توزیع احتمالات گسترده‌تر است و در محدوده وسیع‌تری توزیع شده است. این نشان می‌دهد که مدل در شناسایی این کلاس مشکل بیشتری دارد و اعتماد کمتری به پیش‌بینی‌های منفی دارد.

در شکل ۱۴، توزیع احتمالات برای مدل Boosting Gradient نمایش داده شده است. توزیع مشابه Forest Random است اما برای کلاس تأیید شده، تمرکز بیشتری در مقادیر بالاتر وجود دارد که با Recall بالاتر (96.47%) مطابقت دارد. برای کلاس رد شده، توزیع همچنان گسترده است که نشان می‌دهد مدل در شناسایی این کلاس مشکل دارد. هر دو مدل اعتماد متوسطی به پیش‌بینی‌های خود دارند. مدل‌ها در شناسایی کلاس مثبت بهتر عمل می‌کنند اما در شناسایی کلاس منفی مشکل دارند. این می‌تواند به دلیل عدم تعادل کلاس یا قدرت پیش‌بینی پایین ویژگی‌ها باشد.

۷.۸.۵ خلاصه مقایسه مدل‌ها

در این پروژه، Gradient Boosting با F1-Score بالاتر (0.8039 در مقابل 0.7872) و Recall بالاتر (96.47%) عملکرد بهتری داشت که نشان می‌دهد این مدل در شناسایی وام‌های خوب عملکرد عالی دارد. همچنین، Random Forest با استفاده از `class_weight='balanced'` به Precision بالاتری (71.84% در مقابل 68.91%) دست یافت که برای سیستم وام بسیار مهم است چون باعث کاهش Positive False می‌شود. این نشان می‌دهد که انتخاب مدل بهینه به معیار ارزیابی مورد نظر بستگی دارد: اگر F1-Score و Recall مهم باشند، Gradient



شکل ۱۴: توزیع احتمالات پیش‌بینی برای مدل Boosting Gradient

Boosting بهتر است، اما اگر Precision مهم باشد (که برای سیستم وام اینطور است)، Random Forest با `class_weight='balanced'` انتخاب بهتری است. هر دو مدل عملکرد قابل قبولی دارند و برای کاربردهای واقعی مناسب هستند. برای بهبود بیشتر، می‌توان از روش‌های پیشرفته‌تر مدیریت عدم تعادل کلاس مانند SMOTE استفاده کرد، آستانه تصمیم‌گیری را تنظیم کرد، یا از ویژگی‌های مهندسی شده استفاده کرد.

۶ نتیجه‌گیری

این پروژه شامل چهار بخش اصلی بود که هر کدام جنبه‌های مختلفی از یادگیری ماشین و یادگیری تقویتی را پوشش می‌دهند. در سوال ۱، انتخاب ویژگی با استفاده از الگوریتم‌های PSO و GA انجام شد و هر دو الگوریتم ویژگی Credit_History را به عنوان مهم‌ترین ویژگی انتخاب کردند که با دقت 80.67% همراه بود. در سوال ۲، خوشه‌بندی داده‌ها با استفاده از سه الگوریتم مختلف انجام شد و DBSCAN با امتیاز Silhouette برابر 0.5296 بهترین عملکرد را داشت. در سوال ۳، بررسی نظری الگوریتم یادگیری Q انجام شد و معادله، پارامترها و مسائل همگرایی مورد بررسی قرار گرفت. در پروژه نهایی، پیاده‌سازی کامل خط لوله یادگیری ماشین انجام شد و Gradient Boosting با F1-Score برابر 0.8039 بهترین عملکرد را داشت.

منابع

[۱] <https://scikit-learn.org/> Documentation: Scikit-learn

[2] PySwarms Documentation: <https://pyswarms.readthedocs.io/>

[3] DEAP Documentation: <https://deap.readthedocs.io/>

[4] Sutton, R. S., & Barto, A. G. (2018). Reinforcement learning: An introduction. MIT press.